通用 LVDS

用户手册
V1.0

513 所测试装备事业部

2020 年

用户须知

文档信息

软件版本：2.2.0

发布日期：05.26，2020

文档或软件会不定时更新，可能会产生当前文档与软件不匹配情况，请及时获取最新文档或软件。

联系方式

文档难免有错误和纰漏，欢迎批评和指正，如有更好的建议请联系：

地址：山东省烟台市高新区航天路 513 号山东航天电子技术研究所
测试装备事业部

邮编：264670

电话：0531-6928057，6928142

修订历史

日期	版本	作者	修改内容
2020.05.25	V1.0	wangyl	创建文档

目录

用户须知.....I
文档信息.....I
联系方式.....I
修订历史.....II
目录.....III
1. 概述.....1
1.1 功能.....1
1.2 性能指标及系统要求.....1
1.2.1 性能指标.....1
1.2.2 支持系统.....2
1.3 库文件.....2
1.4 驱动安装.....2
1.5 驱动卸载.....9
1.6 进入测试模式.....10
2. 系统架构.....12
3. 数据结构说明.....14
1) LVDS_BUF_SIZE.....16
2) LVDS_DATA_CH1~LVDS_DATA_CH32.....17
3) ST_LVDS_DATA.....18
4) 波特率定义.....18
5) 数据位定义.....19
6) 停止位定义.....19
7) 校验位定义.....19
8) ST_LVDS_CFG.....20
9) ST_LVDS_FRM_CFG.....20
10) ST_PCI_INFO.....21
11) 发送等待时间.....21
12) 数据字宽类型.....21
13) 回调函数类型.....22

4. 函数接口说明 23
01） LVDS_CountCards 23
02） LVDS_Open 23
03） LVDS_Close 23
04） LVDS_Reset 23
05） LVDS_Start 24
06） LVDS_IsOpen 24
07） LVDS_GetInfo 24
08） LVDS_GetStart 24
09） LVDS_GetVer 25
10） LVDS_GetID 25
11） LVDS_GetFpgaDate 25
12） LVDS_GetIntEn 25
13） LVDS_GetFpgaFreq 25
14） LVDS_GetChDataWidth 25
15） LVDS_CheckVer 26
16） LVDS_ClrFIFO 26
17） LVDS_SetOutTime 26
18） LVDS_SetCfg 26
19） LVDS_GetCfg 27
20） LVDS_SetChFrmCfg 27
21） LVDS_GetChFrmCfg 27
22） LVDS_Send 27
23） LVDS_GetChSendBytes 28
24） LVDS_RstChSendBytes 28
25） LVDS_IsSupportRAM 28
26） LVDS_GetRAMSize 28
27） LVDS_SetRAMFrmTime 29
28） LVDS_GetRAMFrmTime 29
29） LVDS_SetRAMFrmData 29
30） LVDS_GetRAMFrmLen 29

31） LVDS_SetRAMFrmTCnt 30
32） LVDS_GetRAMFrmTCnt 30
33） LVDS_RstRAMFrmHadTCnt 30
34） LVDS_GetRAMFrmHadTCnt 30
35） LVDS_StartRAMSend 30
36） LVDS_GetRAMSta 31
37） LVDS_QueryStart 31
38） LVDS_QueryStop 31
39） LVDS_GetCount 31
40） LVDS_ReadData 32
41） LVDS_IntStart 32
42） LVDS_IntStop 32
43） LVDS_ReadDWord 32
44） LVDS_WriteDWord 33
5. 操作流程与示例 34
5.1 发送流程 34
5.1.1 普通发送流程 34
5.1.2RAM 发送流程 35
5.2 发送示例 38
5.3 接收流程 38
5.4 接收示例 40
6. 应用程序说明 42
6.1 软件运行 42
6.2 接收功能 44
6.3 发送功能 45
6.3.1 文件发送 45
6.3.2 帧发送 46
6.4 日志显示 47
6.5 基本读写接口 47
6.6 调试接口 47
6.7 帧计数判断 49

6.8 程序控制 49
6.9 软件版本 49
常见问题 51
1.板卡驱动安装不成功？ 51
2.打开测试软件识别不到板卡？ 51
3.各功能不好用或版本校验失败？ 51

1. 概述

本手册适用于测试装备事业部研发的所有使用**通用 LVDS 协议**的板卡，包括但不限于 DMMS-CPCI-0G022、DMMS-CPCI-0G102、DMMS-PXI-0021、DMMS-PXI-0022、DMMS-PXI-0031、DMMS-PXI-0032。

本手册是对提供的通用 LVDS 驱动、动态库和板卡测试软件的通信机制、函数接口、收发操作流程、软件使用进行描述的说明性文档。

*注意：虽然命名为 LVDS，但本手册也适用于 422 接口，即使是 422 板卡，只要底层 FPGA 采用**通用 LVDS 协议**就可以使用配套的通用 LVDS 动态库接口。*

1.1 功能

本板卡最多可提供 32 路 LVDS 通道的数据发送和数据接收功能，支持 LVDS 和串口接口，支持全双工操作（发送和接收可同时进行）。

本板卡提供配套的驱动和动态库接口，只需要通过调用接口函数即可完成板卡功能的控制。

配套的函数接口提供基本的寄存器读写接口，便于后续自行升级扩展。

1.2 性能指标及系统要求

1.2.1 性能指标

- 发送最大速率：取决于实际情况
- 接收最大速率：25MBps（在 FIFO 大小 128KB 情况下）
- 发送最小字节数：1 字节
- 接收最小字节数：1 字节
- 所需内存：不超过 16MB（接收循环缓存实际为 32K*256B）

发送速率最大速率取决于实际情况是因为不同波特率会影响发送速率，不同发送模式（普通模式和 RAM 模式）最大发送速率也不同。

接收最大速率受 FPGA 程序每路接收 FIFO 大小的影响，FIFO 越大速率越高，

不同板卡 FIFO 大小不同从而导致最大速率不同。另外，最大速率是指接收速率的总和，如只有 1 路则该路最大接收速率为 25MBps，如果同时 2 路则每路最大接收速率为 12.5MBps。

板卡所需内存是为了进行数据接收缓存而由驱动固定申请的系统内存大小。

1.2.2 支持系统

- Windows XP 32 位
- Windows 7 32 位
- Windows 7 64 位
- Windows 10 64 位

以上列出的为经过测试的系统，对于其他 Windows 系统理论上也支持。

1.3 库文件

本板卡提供的配套库文件如表 1-1 所示，其中库文件除 LVDS_DLL.h 外，其余均区分 32 位和 64 位版本。

表 1-1 库文件列表

版本	名称	类型	说明
32/64 位版本	LVDS.inf/LVDS_x64.inf	INF 文件	驱动程序的描述文件
	LVDS.sys	驱动文件	驱动程序可执行文件
	LVDS_DLL.h	API 头文件	提供 API 的接口函数和类型
	LVDS_DLL.lib	API 静态库	提供 API 的符号链接
	LVDS_DLL.DLL	API 动态链接库	提供 API 动态链接库

1.4 驱动安装

在安装测试装备事业部提供的 PXI 驱动安装包(PXI_Setup_Win32.exe/PXI_Setup_Win64.exe)时，会自动安装通用 LVDS 驱动，这里讲解手动安装驱动的方法。

下面以 Win7 的 64 位系统下驱动安装方法进行说明（以下图片仅供参考）：

1）将板卡插入机箱的 CPCI/PCI/LVDS/PXI/PXIe 插槽。

2）在设备管理器中发现未识别的“其他设备”，如图 1-1 所示。如果插有多块板卡，可能在设备管理器中不止一个未识别设备，通过右键该设备，选择“属性”，在“详细信息”标签的“硬件 Id”属性下，可以看到该板卡的 VEN(VendorID)和 DEV(DeviceID)，如图 1-2 所示，通过该信息可以确定该板卡种类。

如果当前操作系统未处于测试模式，则需要配置操作系统为测试模式，具体方法查看 1.6 节进入测试模式，对于 32 位操作系统，无需配置。

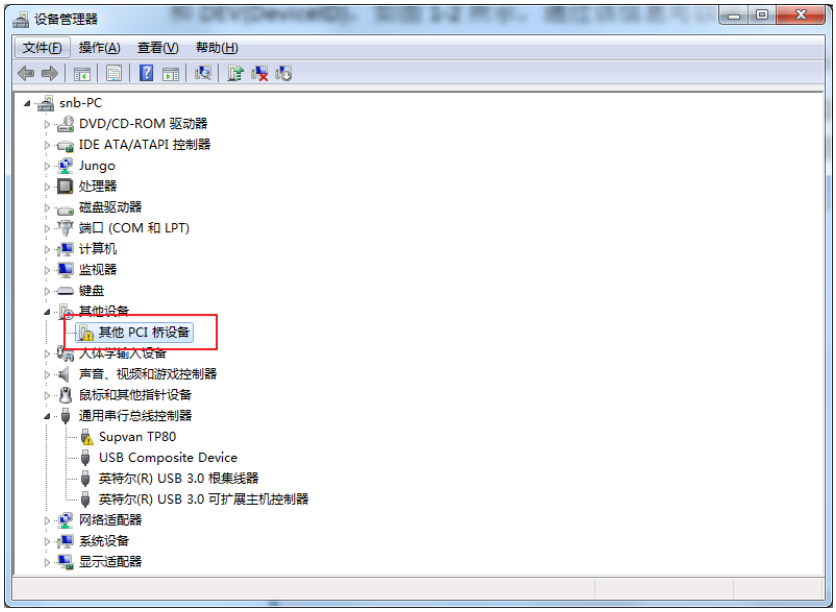


图 1-1 发现“PCI 桥设备”

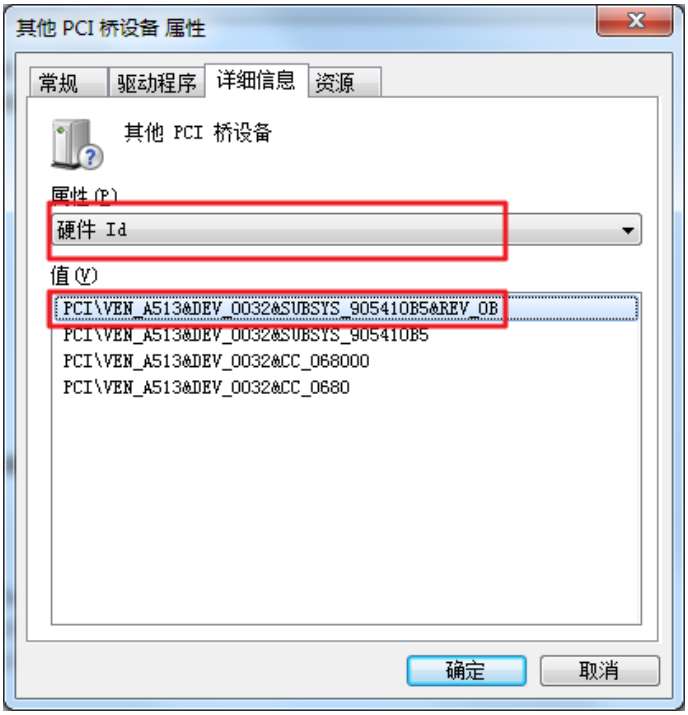


图 1-2 板卡的 VendorID 和 DeviceID

3）右键该板卡，选择“更新驱动程序软件...”，如图 1-3 所示。

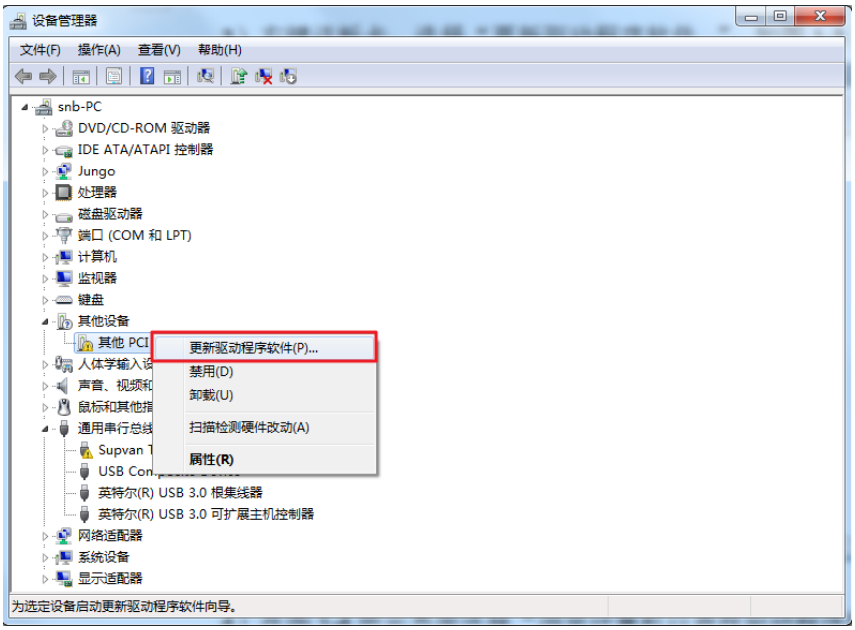


图 1-3 更新驱动程序

4）在图 1-4 所示界面选择“浏览计算机以查找驱动程序软件”。

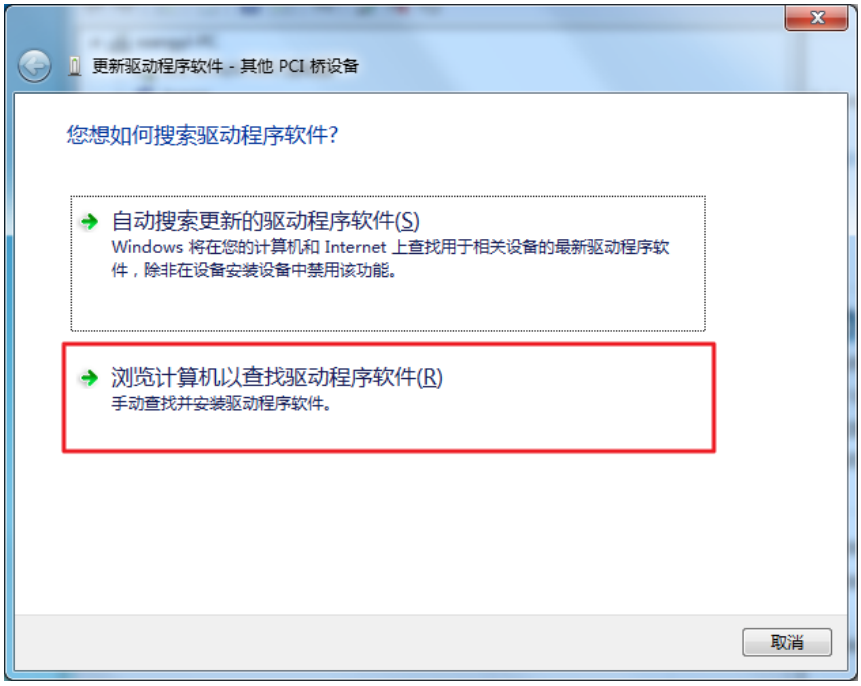


图 1-4 浏览计算机以查找驱动程序软件

5）在图 1-5 所示界面选择“从计算机的设备驱动程序列表中选择”。

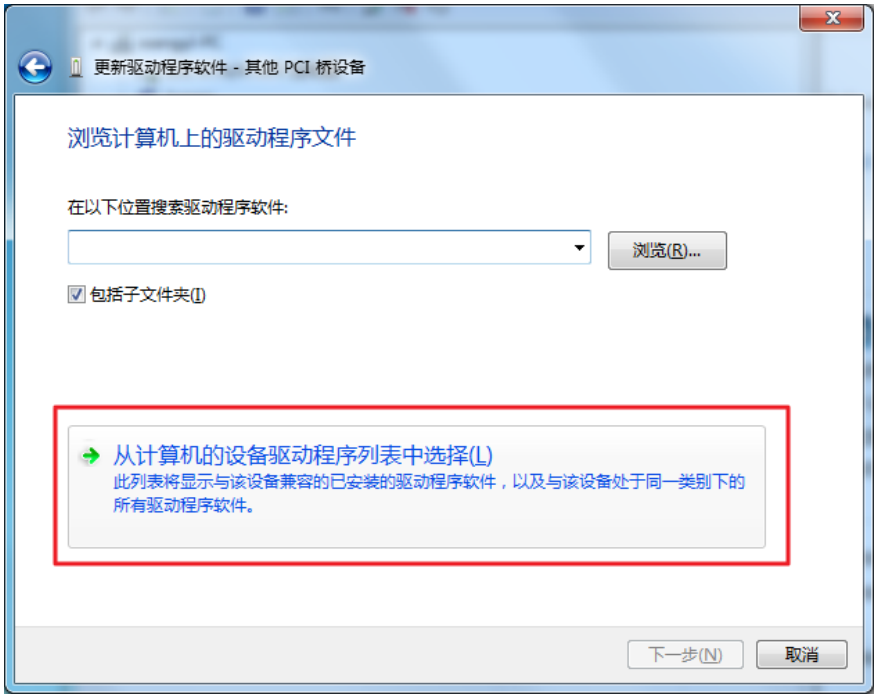


图 1-5 从计算机的设备驱动程序列表中选择

6）在图 1-6 所示界面选择“下一步”。

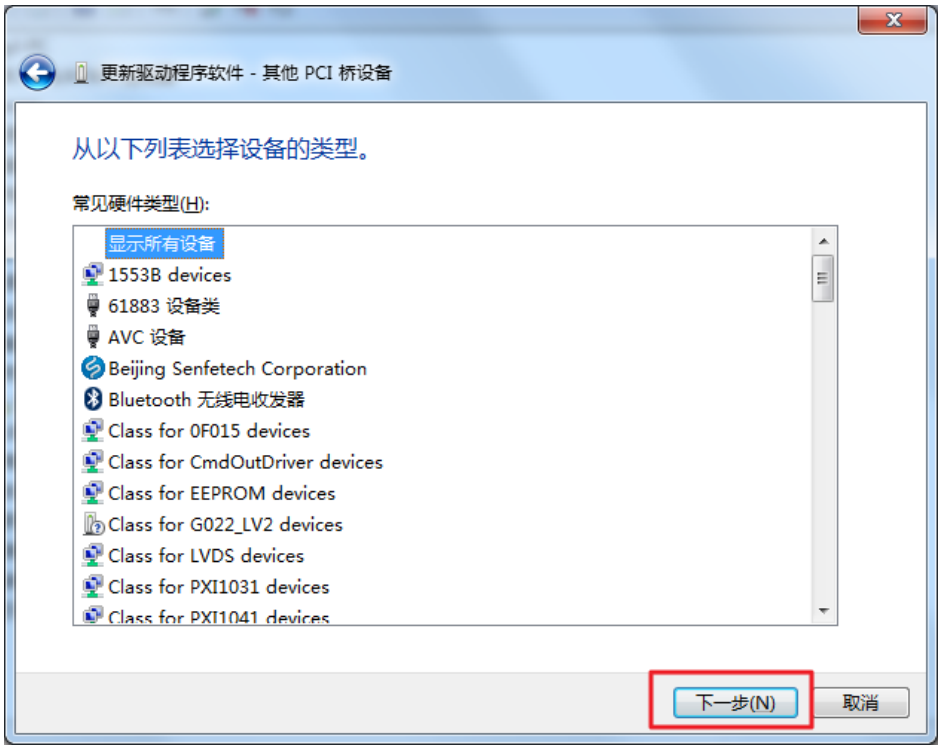


图 1-6 选择下一步

7）在图 1-7 所示界面选择“从磁盘安装...”。

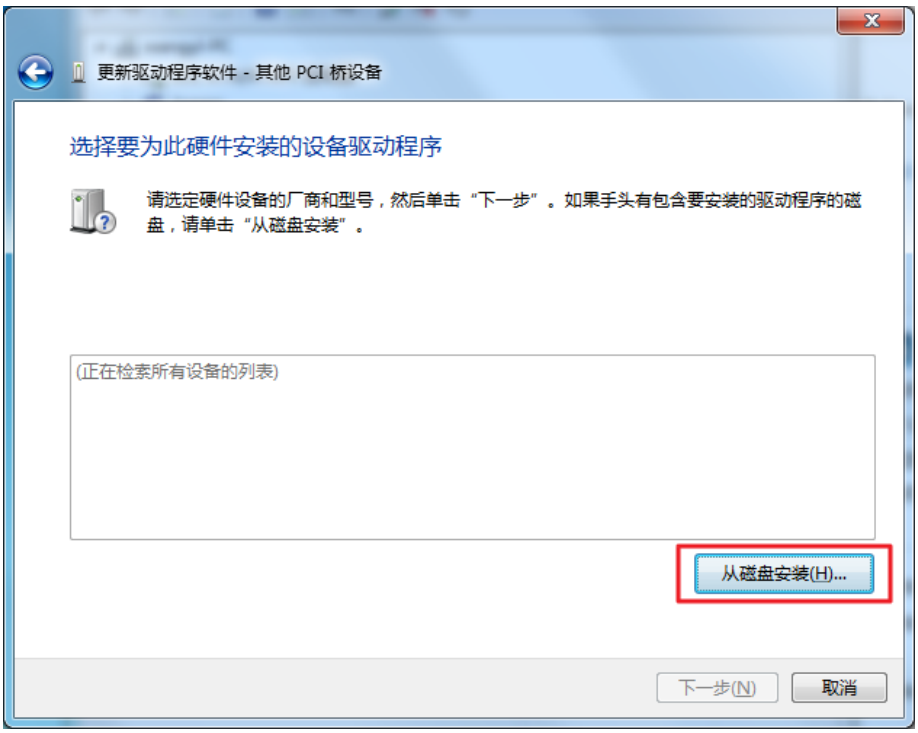


图 1-7 从磁盘安装

8）在图 1-8 所示界面选择“浏览...”，找到 INF 文件所在位置并选择打开，如图 1-9 所示。

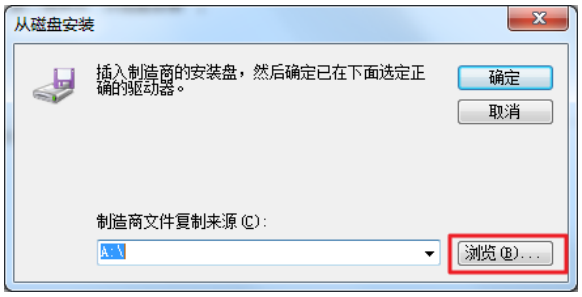


图 1-8 浏览选择 INF 文件

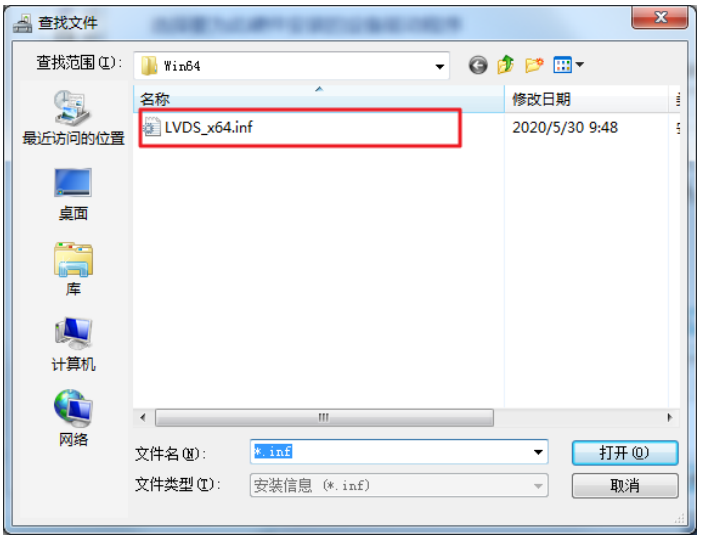


图 1-9 选择 INF 文件后打开

9）在图 1-10 所示界面显示出该板卡的型号名称，点击下一步。

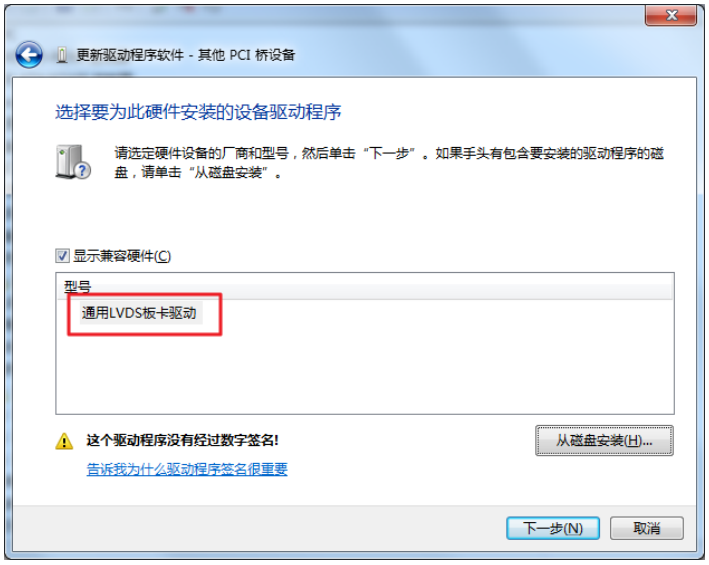


图 1-10 显示该板卡的名称

10）在图 1-11 所示的 Windows 安全界面中选择“始终安装此驱动程序软件”。

图 1-11 始终安装此驱动程序软件

11）驱动安装完成后，出现如图 1-12 所示提示更新驱动程序成功对话框。

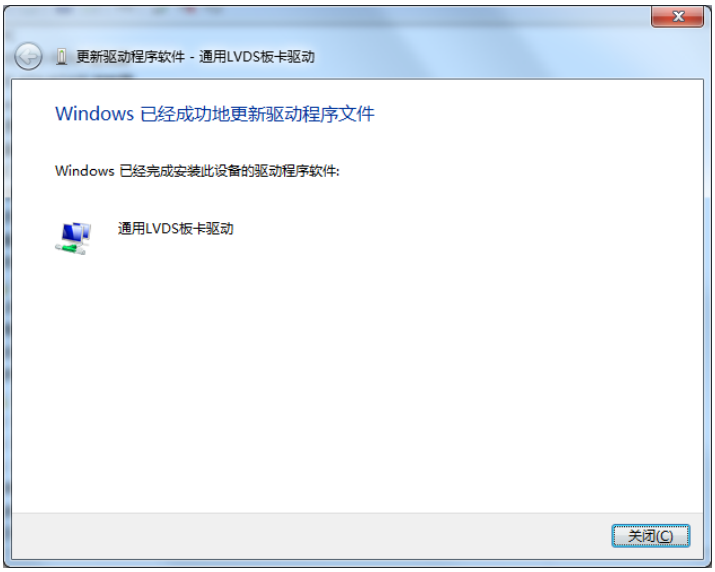


图 1-12 安装完成

12）这时在设备管理器中可以看到该板卡不再以其他设备显示，如图 1-13 所示。

如果为多块该类别板卡安装了驱动，安装完后需要重启下操作系统。

如果驱动安装失败，则该设备图标有黄色叹号（如图 1-14 所示），此时表示驱动安装失败，板卡功能不能正常使用，这时需要确认系统是否处于测试模式（错误信息如图 1-15 所示），或者尝试下重启系统。

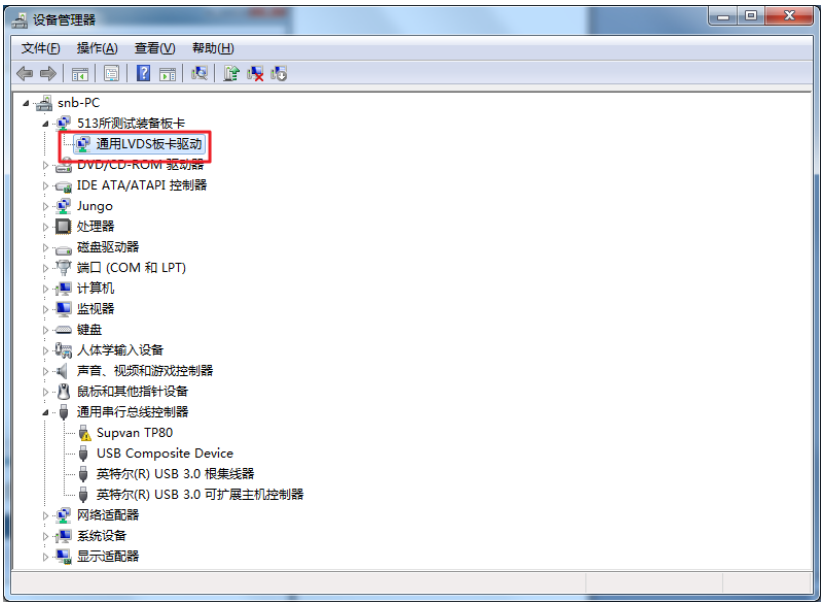


图 1-13 驱动安装成功后的设备管理器

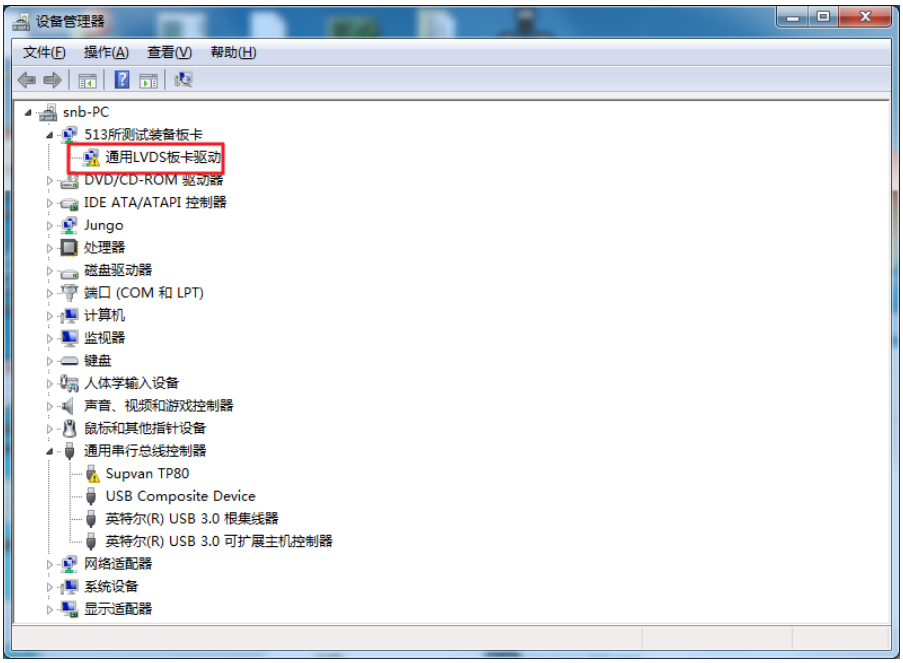


图 1-14 驱动安装失败时的设备管理器

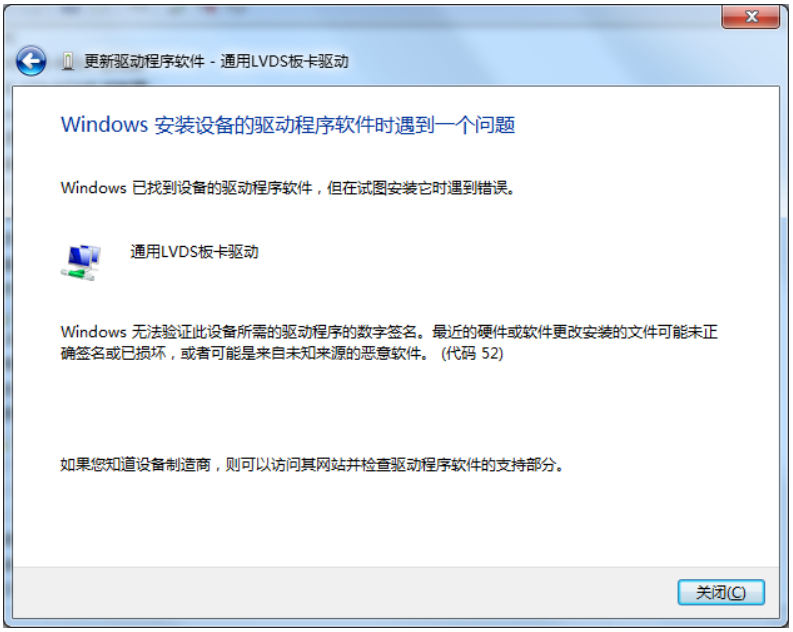


图 1-15 未进入测试模式提示的错误信息

如果设备 ID 与 INF 文件中的 ID 不匹配，会弹出如图 1-16 所示界面，此时选择“是”即可。

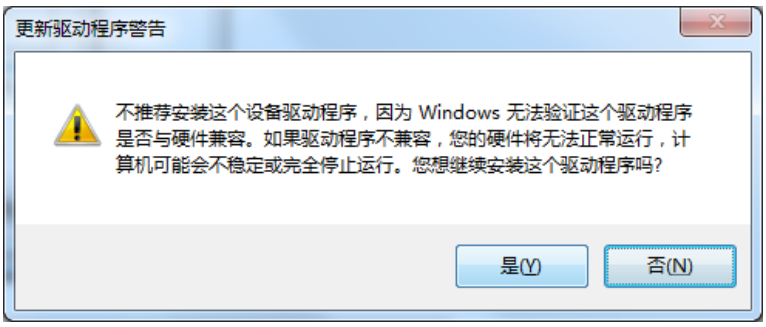


图 1-16 由于资源不足可能的错误信息

1.5 驱动卸载

1）在设备管理器中右键选择需要卸载驱动的板卡设备，在弹出选项中选择“卸载”，如图 1-17 所示。

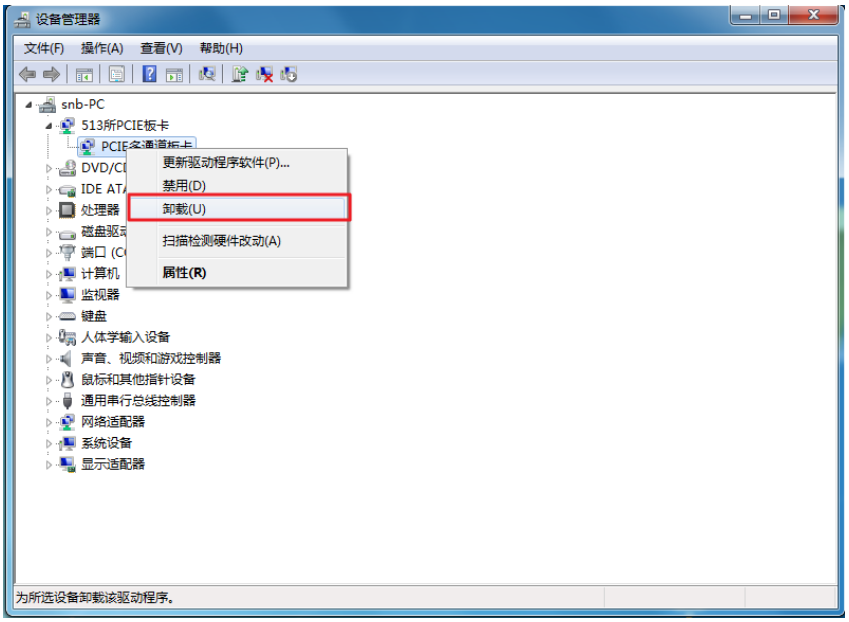


图 1-17 在设备管理器右键选择卸载驱动设备

2）在弹出的图 1-18 所示的对话框中，勾中“删除此设备的驱动程序软件”选项，点击确定即可。

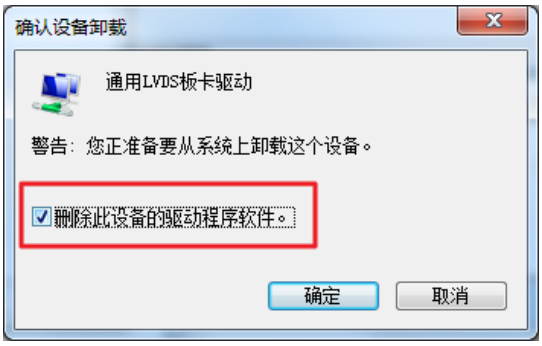


图 1-18 勾中“删除此设备的驱动程序软件”

1.6 进入测试模式

Windows64 位系统下进入测试模式的方法为以管理员身份运行命令提示符(开始->所有程序->附件->命令提示符，如果不是为管理员需要右键选择“以管理员身份运行”)输入“bcdedit /set testsigning on”，回车后提示操作成功完成，如图 1-19 所示，重启操作系统后在操作系统的桌面右下角有“测试模式”的水印，如图 1-20 所示。

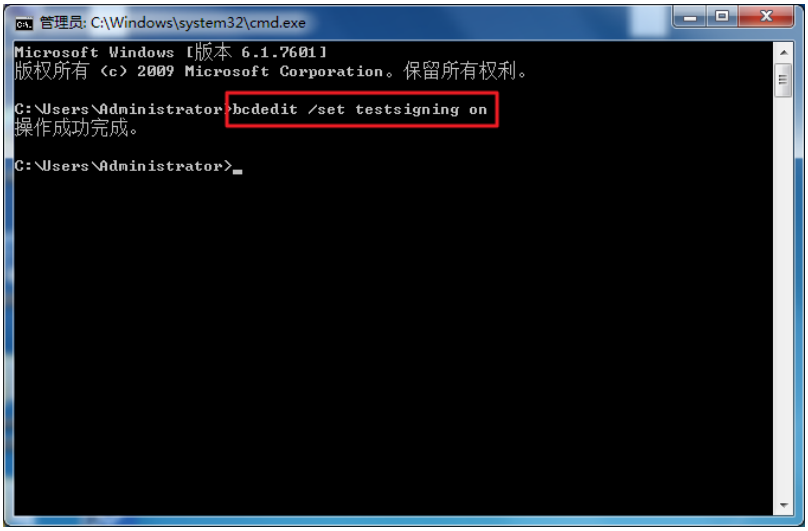


图 1-19 在命令提示符中输入启用测试模式命令

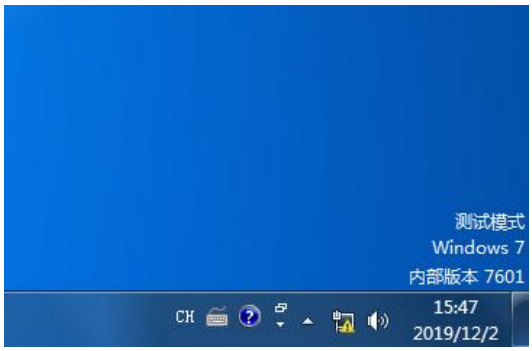


图 1-20 系统右下角的测试模式水印

如果恢复操作系统为非测试模式，方法与进入测试模式类似，只不过是将键入的命令修改为“bcdedit /set testsigning off”，如图 1-21 所示，提示操作成功完成后，重启系统即可。

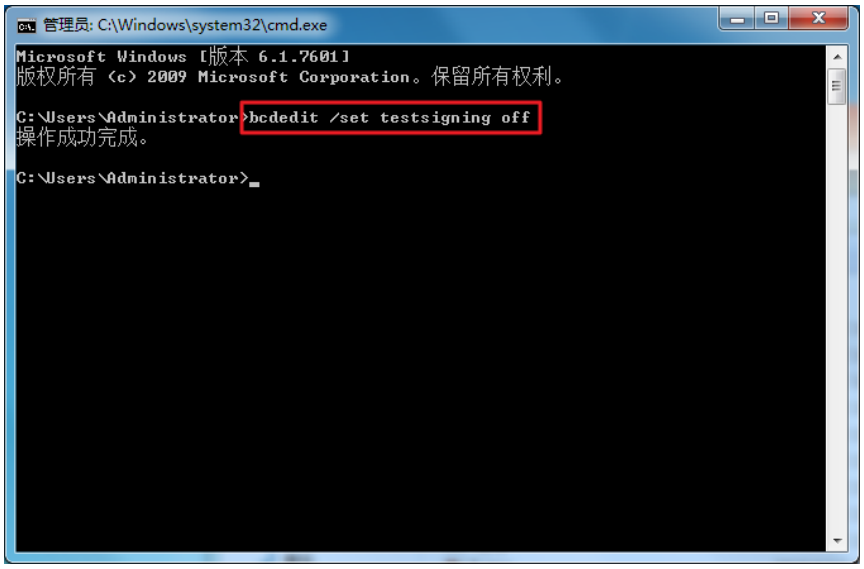


图 1-21 在命令提示符中输入禁用测试模式命令

2. 系统架构

系统的整体架构如图 2-1 所示，按照角色自下往上可划分为 LVDS 板卡、操作系统、驱动程序、动态库和应用软件。

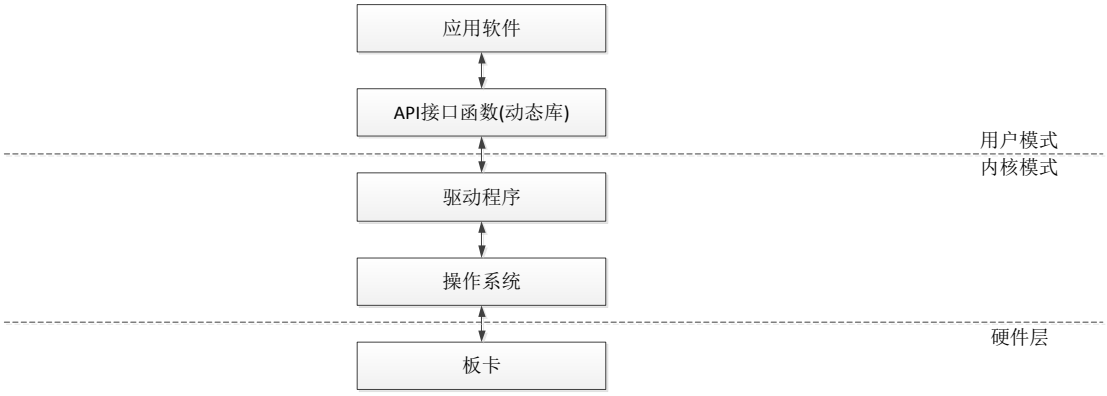


图 2-1 系统架构示意图

其中 LVDS 板卡工作在硬件层，是板卡功能的提供者。操作系统负责提供对硬件的内存映射和端口访问接口，驱动程序使用操作系统的接口实现相应服务，包括缓存分配、中断响应处理、数据读取等。API 接口函数通过驱动程序提供的服务，提供板卡操作、参数配置、数据发送和数据接收等操作。应用软件利用配套提供的动态库接收实现用户实际所需要的数据发送和数据接收功能。

发送原理如图 2-1 所示，应用软件负责数据的拼装，然后调用 API 接口函数通过驱动将发送数据设置到板卡的发送 FIFO，启动发送后，LVDS 板卡将当前的发送状态通过 API 接口函数实时反馈给应用软件。

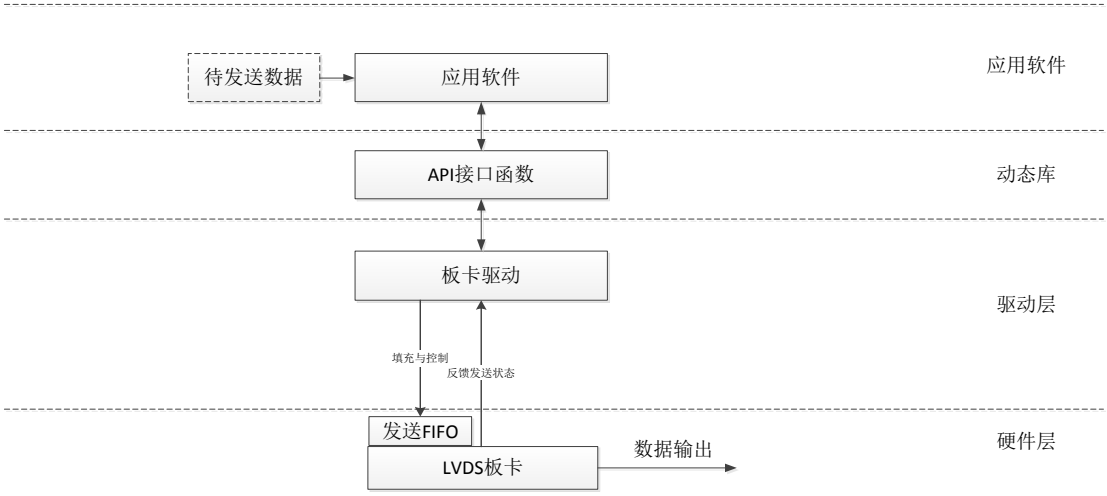


图 2-2 发送原理示意图

接收原理如图 2-1 所示，应用软件通过调用 API 接口函数实现数据接收。驱动层为实现更高的传输速度，为应用软件预留出足够的响应时间，采用循环缓存模式。在 LVDS 板卡的接收 FIFO 达到中断条件后提交中断，由驱动程序对中断进行响应，通过普通读或 DMA 方式将接收 FIFO 数据放入接收循环缓存，并通知应用软件进行读取。同时应用软件不断通过 API 接收函数查询接收循环缓存的可读取数据量进行数据读取。

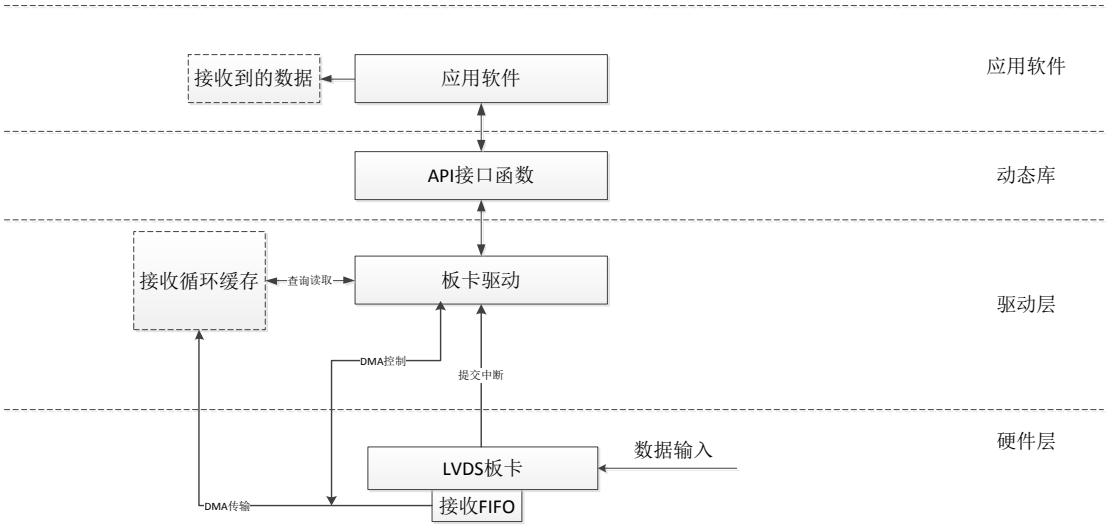


图 2-3 接收原理示意图

3. 数据结构说明

提供的宏、枚举和结构定义如下：

```
//=====================================================================
//以下为类型定义：包括缓存大小、通道号、数据结构体、配置结构体、回调类型
//=====================================================================
#define LVDS_BUF_SIZE            (32*1024) //定义BUF大小为32K

//定义发送和接收通道号,各为32路 ★★★注意通道号LVDS_DATA_CH1为0★★★
enum
{
    LVDS_DATA_CH1 = 0,
    LVDS_DATA_CH2,
    LVDS_DATA_CH3,
    LVDS_DATA_CH4,
    LVDS_DATA_CH5,
    LVDS_DATA_CH6,
    LVDS_DATA_CH7,
    LVDS_DATA_CH8,
    LVDS_DATA_CH9,
    LVDS_DATA_CH10,
    LVDS_DATA_CH11,
    LVDS_DATA_CH12,
    LVDS_DATA_CH13,
    LVDS_DATA_CH14,
    LVDS_DATA_CH15,
    LVDS_DATA_CH16,
    LVDS_DATA_CH17,
    LVDS_DATA_CH18,
    LVDS_DATA_CH19,
    LVDS_DATA_CH20,
    LVDS_DATA_CH21,
    LVDS_DATA_CH22,
    LVDS_DATA_CH23,
    LVDS_DATA_CH24,
    LVDS_DATA_CH25,
    LVDS_DATA_CH26,
    LVDS_DATA_CH27,
    LVDS_DATA_CH28,
    LVDS_DATA_CH29,
    LVDS_DATA_CH30,
    LVDS_DATA_CH31,
    LVDS_DATA_CH32,
    LVDS_DATA_CNT
};

//定义接收数据结构体
typedef struct
{
    UCHAR emType;//数据类型:标识数据从哪个通道接收的 范围为[LVDS_DATA_CH1,LVDS_DATA_CNT)
    ULONG usLen; //数据长度:标识接收的字节数，即标识ucBuf数组中有效数据的长度 范围为[1,LVDS_BUF_SIZE]
    UCHAR ucBuf[LVDS_BUF_SIZE];//接收数据数组:其中数组中的前usLen个字节为接收的有效数据
}ST_LVDS_DATA, *PLVDSDATA;
```

```
//以下为常用波特率定义
enum
{
    LVDS_BAUD_1200   = 1200,
    LVDS_BAUD_2400   = 2400,
    LVDS_BAUD_4800   = 4800,
    LVDS_BAUD_9600   = 9600,
    LVDS_BAUD_19200  = 19200,
    LVDS_BAUD_38400  = 38400,
    LVDS_BAUD_57600  = 57600, //24M时钟有误差
    LVDS_BAUD_115200 = 115200,//24M时钟有误差
    LVDS_BAUD_128000 = 128000,//24M时钟有误差
    LVDS_BAUD_256000 = 256000,//24M时钟有误差
};

//以下为数据位定义
enum
{
    LVDS_DATABITS_1,//1位数据位
    LVDS_DATABITS_2,//2位数据位
    LVDS_DATABITS_3,//3位数据位
    LVDS_DATABITS_4,//4位数据位
    LVDS_DATABITS_5,//5位数据位
    LVDS_DATABITS_6,//6位数据位
    LVDS_DATABITS_7,//7位数据位
    LVDS_DATABITS_8,//8位数据位
};

//以下为停止位定义
enum
{
    LVDS_STOPBITS_10,//1位停止位
    LVDS_STOPBITS_15,//1.5位停止位
    LVDS_STOPBITS_20,//2位停止位
};

//以下为校验位定义
enum
{
    LVDS_PARITY_NO,  //无校验
    LVDS_PARITY_ODD, //奇校验
    LVDS_PARITY_EVEN,//偶校验
};

//定义通道配置参数结构体(如果对于有些成员没用到可忽略)
typedef struct
{
    DWORD  dwBaudRate;//波特率:对应LVDS_BAUD_1200~LVDS_BAUD_256000,对于未列出的波特率也支持
    UCHAR  ucDataBits;//数据位:为LVDS_DATABITS_1~LVDS_DATABITS_8
    UCHAR  ucStopBits;//停止位:为LVDS_STOPBITS_10~LVDS_STOPBITS_20
    UCHAR  ucParity;  //校验位:为LVDS_PARITY_NO~LVDS_PARITY_EVEN
}ST_LVDS_CFG,*PLVDSCFG;
```

```
//定义帧设置结构体
typedef struct
{
    BOOL  bEnable;  //帧使能:TRUE为使能,FALSE为禁止
    DWORD dwFrmLen; //帧长:单位为字节 默认为0x10000
    DWORD dwFrmTime;//帧间隔:单位为FPGA时钟个数(时钟频率可进行查询)
    DWORD dwFrmHead;//帧头:默认为0xEB90EB90
    DWORD dwFrmEnd; //帧尾:默认为0x55AA55AA
}ST_LVDS_FRM_CFG,*PSTLVDSFRMCFG;

#ifndef TYPEDEF_ST_PCI_INFO
#define TYPEDEF_ST_PCI_INFO
typedef struct
{
    ULONG ulBusNo;//总线号
    ULONG ulDevNo;//设备号
    ULONG ulFunc;//功能号
}ST_PCI_INFO,*PPCIINFO;
#endif

//定义发送等待时间类型
enum
{
    LVDS_WAIT_NONE     = 0x00000000,//不等待,立即返回
    LVDS_WAIT_INFINITE = 0xFFFFFFFF,//无限等待
};

//定义数据字宽类型
enum
{
    LVDS_DATAWIDTH_1BYTE,//数据字宽:1字节
    LVDS_DATAWIDTH_2BYTE,//数据字宽:2字节
    LVDS_DATAWIDTH_3BYTE,//数据字宽:3字节
    LVDS_DATAWIDTH_4BYTE,//数据字宽:4字节
};
```

1) LVDS_BUF_SIZE

```
#define LVDS_BUF_SIZE            (32*1024) //定义BUF大小为32K
```

宏定义 LVDS_BUF_SIZE 用于表示单次接收数据的最大字节数。

2) LVDS_DATA_CH1~LVDS_DATA_CH32

```
//定义发送和接收通道号,各为32路 ★★★注意通道号LVDS_DATA_CH1为0★★★
enum
{
    LVDS_DATA_CH1 = 0,
    LVDS_DATA_CH2,
    LVDS_DATA_CH3,
    LVDS_DATA_CH4,
    LVDS_DATA_CH5,
    LVDS_DATA_CH6,
    LVDS_DATA_CH7,
    LVDS_DATA_CH8,
    LVDS_DATA_CH9,
    LVDS_DATA_CH10,
    LVDS_DATA_CH11,
    LVDS_DATA_CH12,
    LVDS_DATA_CH13,
    LVDS_DATA_CH14,
    LVDS_DATA_CH15,
    LVDS_DATA_CH16,
    LVDS_DATA_CH17,
    LVDS_DATA_CH18,
    LVDS_DATA_CH19,
    LVDS_DATA_CH20,
    LVDS_DATA_CH21,
    LVDS_DATA_CH22,
    LVDS_DATA_CH23,
    LVDS_DATA_CH24,
    LVDS_DATA_CH25,
    LVDS_DATA_CH26,
    LVDS_DATA_CH27,
    LVDS_DATA_CH28,
    LVDS_DATA_CH29,
    LVDS_DATA_CH30,
    LVDS_DATA_CH31,
    LVDS_DATA_CH32,
    LVDS_DATA_CNT
};
```

枚举 LVDS_DATA_CH1~LVDS_DATA_CH32 表示 LVDS 板卡的 32 个通道（32 为软件支持的最大通道数量，实际使用通道数量以实际为准），每个通道可同时进行发送和接收操作。

注意：这里的通道号实际值从 0 开始，在实际编程时最好使用定义的枚举来表示通道号，避免使用通道错误。

3) ST_LVDS_DATA

```
//定义接收数据结构体
typedef struct
{
    UCHAR emType;//数据类型:标识数据从哪个通道接收的 范围为[LVDS_DATA_CH1,LVDS_DATA_CNT)
    ULONG usLen; //数据长度:标识接收的字节数，即标识ucBuf数组中有效数据的长度 范围为[1,LVDS_BUF_SIZE]
    UCHAR ucBuf[LVDS_BUF_SIZE];//接收数据数组:其中数组中的前usLen个字节为接收的有效数据
}ST_LVDS_DATA, *PLVDSDATA;
```

ST_LVDS_DATA 结构体定义了数据接收结构体，其中 emType 表示数据来源通道，范围为 LVDS_DATA_CH1~LVDS_DATA_CH32，usLen 表示接收的字节数，也即数组 ucBuf 中有效数据长度，ucBuf 为存放接收数据数组，数组大小为 LVDS_BUF_SIZE，有效数据字节数由 usLen 指定。

4) 波特率定义

```
//以下为常用波特率定义
enum
{
    LVDS_BAUD_1200   = 1200,
    LVDS_BAUD_2400   = 2400,
    LVDS_BAUD_4800   = 4800,
    LVDS_BAUD_9600   = 9600,
    LVDS_BAUD_19200  = 19200,
    LVDS_BAUD_38400  = 38400,
    LVDS_BAUD_57600  = 57600, //24M时钟有误差
    LVDS_BAUD_115200 = 115200,//24M时钟有误差
    LVDS_BAUD_128000 = 128000,//24M时钟有误差
    LVDS_BAUD_256000 = 256000,//24M时钟有误差
};
```

波特率枚举类型定义了常用的波特率值，对于未列出波特率也支持，使用未列出波特率可直接使用常数设置，比如 512000 表示波特率 512000。

对于支持波特率的板卡，不同板卡可能具有不同的默认波特率，可通过函数 LVDS_GetCfg 进行查询。

注意：对于不能被 FPGA 时钟频率整除的波特率会有误差，例如 FPGA 时钟为 24MHz，则使用波特率 57600 会有误差。FPGA 时钟频率可通过函数 LVDS_GetFpgaFreq 得到，单位为 Hz。

5) 数据位定义

```
//以下为数据位定义
enum
{
    LVDS_DATABITS_1,//1位数据位
    LVDS_DATABITS_2,//2位数据位
    LVDS_DATABITS_3,//3位数据位
    LVDS_DATABITS_4,//4位数据位
    LVDS_DATABITS_5,//5位数据位
    LVDS_DATABITS_6,//6位数据位
    LVDS_DATABITS_7,//7位数据位
    LVDS_DATABITS_8,//8位数据位
};
```

数据位枚举定义了常用的数据位，支持 1 位~8 位数据位。

对于板卡默认的数据位数可通过函数 LVDS_GetCfg 进行查询。

6) 停止位定义

```
//以下为停止位定义
enum
{
    LVDS_STOPBITS_10,//1位停止位
    LVDS_STOPBITS_15,//1.5位停止位
    LVDS_STOPBITS_20,//2位停止位
};
```

停止位枚举定义了常用的停止位，包括 1 位、1.5 位、2 位停止位。

对于板卡默认的停止位数可通过函数 LVDS_GetCfg 进行查询。

7) 校验位定义

```
//以下为校验位定义
enum
{
    LVDS_PARITY_NO,  //无校验
    LVDS_PARITY_ODD, //奇校验
    LVDS_PARITY_EVEN,//偶校验
};
```

校验位枚举类型定义了常用的校验方式，包括无校验、奇校验、偶校验。

对于板卡默认的校验位可通过函数 LVDS_GetCfg 进行查询。

8) ST_LVDS_CFG

```
//定义通道配置参数结构体(如果对于有些成员没用到可忽略)
typedef struct
{
    DWORD  dwBaudRate;//波特率:对应LVDS_BAUD_1200~LVDS_BAUD_256000,对于未列出的波特率也支持
    UCHAR  ucDataBits;//数据位:为LVDS_DATABITS_1~LVDS_DATABITS_8
    UCHAR  ucStopBits;//停止位:为LVDS_STOPBITS_10~LVDS_STOPBITS_20
    UCHAR  ucParity;  //校验位:为LVDS_PARITY_NO~LVDS_PARITY_EVEN
}ST_LVDS_CFG,*PLVDSCFG;
```

结构体 ST_LVDS_CFG 用于进行通道参数配置或查询。其中成员 dwBaudRate 为波特率，取值范围为 LVDS_BAUD_1200~LVDS_BAUD_256000，也可以为其它支持的波特率数值。成员 ucDataBits 表示数据位，取值范围为 LVDS_DATABITS_1~LVDS_DATABITS_8。成员 ucStopBits 表示停止位，取值范围为 LVDS_STOPBITS_10~LVDS_STOPBITS_20。成员 ucParity 表示校验位，取值范围为 LVDS_PARITY_NO~LVDS_PARITY_EVEN。

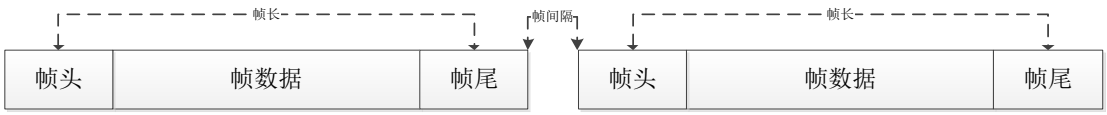
注意：该结构体为通道参数最大包络，对于不需要设置或 FPGA 不支持的参数，设置无效。例如 LVDS 板卡只支持波特率设置，不需要设置数据位、停止位、校验位，则此时成员 ucDataBits、ucStopBits、ucParity 的值将被 FPGA 忽略。具体哪些参数需要或可设置，需要咨询 FPGA 人员或项目负责人。

9) ST_LVDS_FRM_CFG

```
//定义帧设置结构体
typedef struct
{
    BOOL  bEnable;  //帧使能:TRUE为使能,FALSE为禁止
    DWORD dwFrmLen; //帧长:单位为字节 默认为0x10000
    DWORD dwFrmTime;//帧间隔:单位为FPGA时钟个数(时钟频率可进行查询)
    DWORD dwFrmHead;//帧头:默认为0xEB90EB90
    DWORD dwFrmEnd; //帧尾:默认为0x55AA55AA
}ST_LVDS_FRM_CFG,*PSTLVDSFRMCFG;
```

结构体 ST_LVDS_FRM_CFG 用于进行帧设置或帧配置信息查询。其中成员 bEnable 表示帧使能，TRUE 为使能，FALSE 为禁止。成员 dwFrmLen 表示帧长，单位为字节。成员 dwFrmTime 表示帧间隔，单位为 FPGA 时钟个数，FPGA 时钟频率可通过函数 LVDS_GetFpgaFreq 查询。成员 dwFrmHead 表示帧头，占用 4 个字节。成员 dwFrmEnd 表示帧尾，占用 4 个字节。

帧结构示意如图 3-1 所示。



3-1 帧结构示意图

10) ST_PCI_INFO

```
typedef struct
{
    ULONG ulBusNo;//总线号
    ULONG ulDevNo;//设备号
    ULONG ulFunc;//功能号
}ST_PCI_INFO,*PPCIINFO;
```

结构体 ST_PCI_INFO 用于查询板卡所在插槽的信息，ulBusNo 表示所在总线号，ulDevNo 表示设备号，ulFunc 表示功能号。

11) 发送等待时间

```
//定义发送等待时间类型
enum
{
    LVDS_WAIT_NONE     = 0x00000000,//不等待,立即返回
    LVDS_WAIT_INFINITE = 0xFFFFFFFF,//无限等待
};
```

发送等待时间用于定义在发送时等待发送完成的最大等待时间，避免出现该次发送未完成，又开始下次数据发送，导致数据错误的情况。LVDS_WAIT_NONE 表示不进行发送完成判断，设置完数据后立即返回。LVDS_WAIT_INFINITE 表示一直等待，直到数据发送完成。其他值表示等待时间为有限时间，比如 5000 表示最大等待时间为 5000ms。

12) 数据字宽类型

```
//定义数据字宽类型
enum
{
    LVDS_DATAWIDTH_1BYTE,//数据字宽:1字节
    LVDS_DATAWIDTH_2BYTE,//数据字宽:2字节
    LVDS_DATAWIDTH_3BYTE,//数据字宽:3字节
    LVDS_DATAWIDTH_4BYTE,//数据字宽:4字节
};
```

数据字宽用于约定动态库和驱动与 FPGA 数据传输的字长单位，只提供查询

操作。例如发送字宽为 1 字节，则动态库与 FPGA 单次传输数据的字宽为 1 字节。例如接收字宽为 4 字节，则驱动与 FPGA 单次传输数据的字宽为 4 字节。

13) 回调函数类型

```
//定义接收回调函数类型
typedef void (*LVDS_CALLBACK)(UCHAR ucCard,  PLVDSDATA pstData, DWORD dwCount); //ucCard为板卡号,pstData为结构体数组指针 dwCount为结构体数组的个数
```

回调函数类型用于声明进行数据接收时的回调函数，参数 ucCard 表示接收到数据的板卡号，参数 PLVDSDATA 为接收数据的结构体数据，参数 dwCount 表示结构体数组大小。

4. 函数接口说明

提供的函数接口如下：

01）LVDS_CountCards

函数原型：intLVDS_CountCards();
说明：该函数用于统计LVDS板卡数目。
参数：无。
返回值：板卡数。为0表示不存在LVDS板卡。
注意：
1）统计的正确安装驱动后的板卡数量，最多支持32块板卡。
2）该函数实现原理为遍历所有板卡进行打开操作来得到板卡数量，因此如果有板卡已经调用LVDS_Open函数打开后再进行LVDS_CountCards调用将会导致之前的板卡功能异常。该问题已有解决方案，将在后续升级进行解决。

02）LVDS_Open

函数原型：BOOL LVDS_Open (UCHAR ucCard);
说明：该函数用于打开LVDS板卡。在利用LVDS板进行操作前需要先调用此函数打开板卡。
参数：1）ucCard为板卡索引号，从0开始，最大为31。
返回值：TRUE　操作成功
　　FALSE操作失败
注意：板卡在已经打开的情况下再打开会失败。当有多块板卡时，板卡对应索引号取决于实际机箱，板卡索引号不一定按照所插位置依次递增。

03）LVDS_Close

函数原型：BOOL LVDS_Close(UCHAR ucCard);
说明：该函数用于关闭LVDS板卡。
参数：1）ucCard为板卡索引号。
返回值：TRUE　操作成功
　　FALSE操作失败
注意:关闭没有打开的板卡会返回失败。在打开板卡后,在退出应用软件前需要调用该函数释放资源。

04）LVDS_Reset

函数原型：BOOL LVDS_Reset(UCHAR ucCard);
说明：进行板卡复位，恢复板卡FPGA状态为刚上电状态。

参数：1）ucCard为板卡索引号。

返回值：TRUE　操作成功

FALSE操作失败

05）LVDS_Start

函数原型：BOOL LVDS_Start(UCHAR ucCard,BOOL bStart);

说明：进行启动停止操作，只有处于启动状态板卡才处于工作状态。

参数：1）ucCard为板卡索引号。

2）bStart为TRUE启动板卡，为FALSE停止板卡。

返回值：TRUE　操作成功

FALSE操作失败

说明：该函数为满足有些项目中要求板卡上电后不进行任何工作或必须进行某些配置才可工作，该函数可进行很好的FPGA功能分离与调试，并不是多余函数。

06）LVDS_IsOpen

函数原型：BOOL LVDS_IsOpen(UCHAR ucCard);

说明：查询当前板卡是否打开。

参数：1）ucCard为板卡索引号。

返回值：TRUE　是

FALSE否

07）LVDS_GetInfo

函数原型：BOOL LVDS_GetInfo(UCHAR ucCard,PPCIINFO pstInfo);

说明：该函数用于获取板卡所在插槽信息。

参数：1）ucCard为板卡索引号。

2）pstInfo为板卡所在插槽信息，包括总线号、设备号和功能号。

返回值：TRUE　操作成功

FALSE操作失败

注意：可通过总线号来区分不同的板卡。

08）LVDS_GetStart

函数原型：DWORDLVDS_GetStart(UCHAR ucCard);

说明：该函数用于获取板卡启动状态。

参数：1）ucCard为板卡索引号。

返回值：0x00：未启动或未打开板卡，0x01：启动状态。

09）LVDS_GetVer

函数原型：DWORD**LVDS_GetVer**(UCHAR ucCard);
说明：该函数用于获取板卡版本信息。
参数：1）ucCard为板卡索引号。
返回值:版本信息,高16位为板卡类,低16位为版本号,例如0x00320202,表示板卡DeviceID为0x0032,程序版本为V02.02。

10）LVDS_GetID

函数原型：DWORD**LVDS_GetID**(UCHAR ucCard);
说明：该函数用于获取设备ID。
参数：1）ucCard为板卡索引号。
返回值：32位板卡ID，例如0xA5130031。

11）LVDS_GetFpgaDate

函数原型：DWORD**LVDS_GetFpgaDate**(UCHAR ucCard);
说明：该函数用于FPGA编译日期。
参数：1）ucCard为板卡索引号。
返回值：编译日期，采用BCD码，例如0x20200708表示2020年07月08日。

12）LVDS_GetIntEn

函数原型：DWORD**LVDS_GetIntEn**(UCHAR ucCard);
说明：该函数用于获取接收使能状态。
参数：1）ucCard为板卡索引号。
返回值：32个接收通道的中断使能状态，比特位为1表示使能，为0表示禁止，BIT0表示通道LVDS_DATA_CH1状态，依次类推。

13）LVDS_GetFpgaFreq

函数原型：DWORD**LVDS_GetFpgaFreq**(UCHAR ucCard);
说明：该函数用于获取该板卡FPGA的时钟频率。
参数：1）ucCard为板卡索引号。
返回值：时钟频率值，单位为Hz。

14）LVDS_GetChDataWidth

函数原型：BOOL**LVDS_GetChDataWidth**(UCHAR ucCard,BOOL bRcv,BYTE byCh,BYTE *pbyWidth);

说明：该函数用于查询指定通道发送或接收数据位宽。

参数：1）ucCard为板卡索引号。

2）bRcv为TRUE表示接收通道，FALSE表示发送通道。

3）byCh表示通道号。

4）pbyWidth用于带出位宽值，值为LVDS_DATA_WIDTH_1BYTE~LVDS_DATA_WIDTH_4BYTE。

返回值：TRUE　操作成功

FALSE操作失败

15）LVDS_CheckVer

函数原型：BOOL**LVDS_CheckVer**(UCHAR ucCard);

说明：该函数用于校验动态库版本是否匹配FPGA程序版本和驱动版本。

参数：1）ucCard为板卡索引号。

返回值：TRUE　匹配

FALSE不匹配

16）LVDS_ClrFIFO

函数原型：BOOL **LVDS_ClrFIFO**(UCHAR ucCard);

说明：该函数用于清空FIFO以及驱动中已经接收的数据。

参数：1）ucCard为板卡索引号。

返回值：TRUE　操作成功

FALSE操作失败

17）LVDS_SetOutTime

函数原型：BOOL **LVDS_SetOutTime**(UCHAR ucCard,DWORD dwTimeMs);

说明：设置接收中断的超时时间。

参数：1）ucCard为板卡索引号。

2）dwTimeMs为设置的超时时间，单位为ms。

返回值：TRUE　操作成功

FALSE操作失败

18）LVDS_SetCfg

函数原型：BOOL **LVDS_SetCfg** (UCHAR ucCard,BOOL bRcv,BYTE byCh,PLVDSCFG pstCfg);

说明：设置指定通道发送或接收参数。

参数：1）ucCard为板卡索引号。

2）bRcv表示设置发送还是接收，为TRUE表示接收，为FALSE表示发送。

3）byCh表示通道号。

4）pstCfg为设置的通道参数，包括波特率、数据位、停止位和校验位。

返回值：TRUE　操作成功

FALSE操作失败

说明：对于有些板卡可能不支持或不使用所有参数，不支持的参数将被FPGA忽略，因此可在设置时无需指定不支持的参数。例如LVDS板卡只支持波特率设置，则可在结构体中只设置波特率的值，对于其它不支持的参数可设置为0或其他任意值。

19）LVDS_GetCfg

函数原型：BOOL **LVDS_GetCfg** (UCHAR ucCard,BOOL bRcv,BYTE byCh,PLVDSCFG pstCfg);

说明：查询指定通道发送或接收帧配置。

参数：1）ucCard为板卡索引号。

2）bRcv表示查询发送还是接收，为TRUE表示接收，为FALSE表示发送。

3）byCh表示通道号。

4）pstCfg为获取的通道参数，包括波特率、数据位、停止位和校验位。

返回值：TRUE　操作成功

FALSE操作失败

20）LVDS_SetChFrmCfg

函数原型：BOOL **LVDS_SetChFrmCfg** (UCHAR ucCard,BOOL bRcv,BYTE byCh,PSTLVDSFRMCFG pstCfg);

说明：设置指定通道发送或接收帧配置。

参数：1）ucCard为板卡索引号。

2）bRcv表示设置发送还是接收，为TRUE表示接收，为FALSE表示发送。

3）byCh表示通道号。

4）pstCfg为设置的帧配置参数，包括帧使能、帧长、帧间隔、帧头、帧尾。

返回值：TRUE　操作成功

FALSE操作失败

21）LVDS_GetChFrmCfg

函数原型：BOOL **LVDS_GetChFrmCfg** (UCHAR ucCard,BOOL bRcv,BYTE byCh,PSTLVDSFRMCFG pstCfg);

说明：查询指定通道发送或接收帧配置。

参数：1）ucCard为板卡索引号。

2）bRcv表示查询发送还是接收，为TRUE表示接收，为FALSE表示发送。

3）byCh表示通道号。

4）pstCfg为获取的帧配置参数，包括帧使能、帧长、帧间隔、帧头、帧尾。

返回值：TRUE　操作成功

FALSE操作失败

22）LVDS_Send

函数原型：DWORD **LVDS_Send** (UCHAR ucCard,BYTE byCh,BYTE*pbyBuf, DWORD dwBytes,DWORD

dwWaitTime);

说明：指定通道进行数据发送，默认为阻塞时，只有发送完成该函数才会返回。

参数：1）ucCard为板卡索引号。

2）byCh表示通道号。

3）pbyBuf为待发送的数据地址。

4）dwBytes为待发送字节数。

5）dwWaitTime为等待时间，LVDS_WAIT_NONE为不等待，LVDS_WAIT_INFINITE为无限等待，其他值为有限等待，单位为ms。

返回值：实际发送的数据字节数。

23）LVDS_GetChSendBytes

函数原型：DWORD**LVDS_GetChSendBytes**(UCHAR ucCard,BYTE byCh);

说明：用于查询指定通道FPGA实际已经发送总字节数。

参数：1）ucCard为板卡索引号。

2）byCh表示通道号。

返回值：实际已经发送总字节数，最大到4GB，超出会自动重置。

注意：调用LVDS_Reset或LVDS_RstChSendBytes会重置该计数。

24）LVDS_RstChSendBytes

函数原型：BOOL**LVDS_RstChSendBytes**(UCHAR ucCard,BYTE byCh);

说明：用于重置指定通道FPGA实际已经发送总字节数。

参数：1）ucCard为板卡索引号。

2）byCh表示通道号。

返回值：TRUE　操作成功

FALSE操作失败

25）LVDS_IsSupportRAM

函数原型：BOOL**LVDS_IsSupportRAM**(UCHAR ucCard);

说明：查询是否支持RAM方式发送。只有新版本支持RAM发送方式。

参数：1）ucCard为板卡索引号。

返回值：TRUE　支持

FALSE不支持

26）LVDS_GetRAMSize

函数原型：DWORD**LVDS_GetRAMSize** (UCHAR ucCard,BYTE byCh);

说明：查询指定通道RAM大小，单位为字节数。

参数：1）ucCard为板卡索引号。

2）byCh表示通道号。

返回值：返回该通道的RAM空间大小。

27）LVDS_SetRAMFrmTime

函数原型：BOOL LVDS_SetRAMFrmTime (UCHAR ucCard,BYTE byCh, DWORD dwTime);
说明：设置RAM数据发送的帧间隔时间，单位为us。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。
3）dwTime为帧间隔时间，单位为us。
返回值：TRUE　操作成功
FALSE操作失败
注意：该函数内部根据FPGA时钟频率自动将帧间隔时间转换为时钟个数。

28）LVDS_GetRAMFrmTime

函数原型：DWORD LVDS_GetRAMFrmTime(UCHAR ucCard,BYTE byCh);
说明：查询指定通道帧间隔时间，单位为us。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。
返回值：查询的帧间隔时间。

29）LVDS_SetRAMFrmData

函数原型：DWORD LVDS_SetRAMFrmData (UCHAR ucCard,BYTE byCh, BYTE *pbyBuf, DWORD dwFrmLen);
说明：设置指定通道RAM帧数据。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。
3）pbyBuf为待设置数据地址。
4）dwFrmLen为帧长，也即pbyBuf数据字节数。
返回值：实际设置成功字节数。

30）LVDS_GetRAMFrmLen

函数原型：DWORD LVDS_GetRAMFrmLen (UCHAR ucCard,BYTE byCh);
说明：查询指定通道帧数据帧长，单位为字节数。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。
返回值：查询到的帧长，单位为字节数。

31）LVDS_SetRAMFrmTCnt

函数原型：BOOL**LVDS_SetRAMFrmTCnt** (UCHAR ucCard, BYTE byCh,DWORD dwFrmCnt);
说明：设置指定通道发送帧数。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。
3)dwFrmCnt为发送帧数,LVDS_RAM_SEND_INFINITE(0)表示无限发送,其他值为有限发送。
返回值：TRUE　操作成功
FALSE操作失败

32）LVDS_GetRAMFrmTCnt

函数原型：DWORD**LVDS_GetRAMFrmTCnt** (UCHAR ucCard, BYTE byCh);
说明：查询指定通道设置发送帧数。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。
返回值：查询到的设置发送帧数。

33）LVDS_RstRAMFrmHadTCnt

函数原型：BOOL**LVDS_RstRAMFrmHadTCnt**(UCHAR ucCard, BYTE byCh);
说明：重置指定通道实际已发送帧数。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。
返回值：TRUE　操作成功
FALSE操作失败

34）LVDS_GetRAMFrmHadTCnt

函数原型：DWORD**LVDS_GetRAMFrmHadTCnt**(UCHAR ucCard, BYTE byCh);
说明：查询指定通道实际已发送帧数。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。
返回值：指定通道实际已发送的帧数，单位为帧数。

35）LVDS_StartRAMSend

函数原型：BOOL**LVDS_StartRAMSend** (UCHAR ucCard, BYTE byCh, BOOL bStart);
说明：启动或停止指定通道RAM方式发送。
参数：1）ucCard为板卡索引号。
2）byCh表示通道号。

2）bStart为TRUE表示启动发送，为FALSE表示停止发送。

返回值：TRUE　操作成功

FALSE操作失败

36）LVDS_GetRAMSta

函数原型：BOOL**LVDS_GetRAMSta** (UCHAR ucCard,BYTE byCh);

说明：查询指定通道RAM发送状态。

参数：1）ucCard为板卡索引号。

2）byCh表示通道号。

返回值：TRUE　正在发送

FALSE未发送

37）LVDS_QueryStart

函数原型：BOOL**LVDS_QueryStart** (UCHAR ucCard);

说明：启动所有通道的数据接收操作，使能接收中断。

参数：1）ucCard为板卡索引号。

返回值：TRUE　操作成功

FALSE操作失败

注意：

1）只有调用该函数驱动才会进行中断处理，应用层才能接收到数据。

2）如果未使能数据接收操作，并且此时板卡处于启动状态，即调用LVDS_Start启动了板卡，则此时FPGA会进行数据接收，但不能将中断提交给驱动，因此应用层不能接收到数据。如果FPGA接收的数据超出接收FIFO大小，将会达到FIFO满状态，FPGA不会再接收新数据，此时如果启动数据接收后需要先进行FIFO重置（调用LVDS_ClrFIFO）。

3）如果未使能数据接收，并且此时板卡处于未启动状态，则此时FPGA也不会进行数据接收操作，即FIFO不会存放数据，也就不会发生接收FIFO数据满的情况。

38）LVDS_QueryStop

函数原型：BOOL **LVDS_QueryStop** (UCHAR ucCard);

说明：停止所有通道的数据接收操作，禁止接收中断。

参数：1）ucCard为板卡索引号。

返回值：TRUE　操作成功

FALSE操作失败

39）LVDS_GetCount

函数原型：DWORD **LVDS_GetRCount**(UCHAR ucCard, DWORD dwWaitMs = 20);

说明：该函数用于查询接收数据结构个数。

参数：1）ucCard为板卡索引号。

返回值：接收缓存中存放的数据结构个数。

注意：返回值不是字节个数，而是数据结构个数。

40）LVDS_ReadData

函数原型：DWORD **LVDS_ReadData** (UCHAR ucCard, PLVDSDATA pstData, DWORD dwCount);

说明：该函数用于读取接收缓存中的数据。

参数：1）ucCard为板卡索引号。

2）pstData为待存放数据的结构数组地址。

2）dwCount为读取数据结构的个数，pstData数组中元素个数至少为dwCount。

返回值：实际读取数据结构个数。

41）LVDS_IntStart

函数原型：BOOL**LVDS_IntStart**(UCHAR ucCard, LVDS_CALLBACK pfCallBack);

说明：回调方式接收数据，注册接收数据回调函数，使能接收中断，在接收到数据时自动调用回调函数。

参数：1）ucCard为板卡索引号。

2）pfCallBack为回调函数指针。

返回值：TRUE　操作成功

FALSE操作失败

注意：该种方式与查询方式（LVDS_GetCount）为读取接收数据的两种方式之一，内部实现机制也是开启线程不断调用LVDS_GetCount进行查询，所以建议使用查询方式，这样更灵活，更好实现自定义。

42）LVDS_IntStop

函数原型：BOOL **LVDS_IntStop**(UCHAR ucCard);

说明：停止回调方式接收数据，禁止接收中断。

参数：1）ucCard为板卡索引号。

返回值：TRUE　操作成功

FALSE操作失败

43）LVDS_ReadDWord

函数原型：DWORD **LVDS_ReadDWord** (UCHAR ucCard , DWORD dwOffset);

说明：该函数提供基本读接口：用于读取指定FPGA寄存器数据。

参数：1）ucCard为板卡索引号。

2）dwOffset为寄存器地址。

返回值：读取到的数据。

注意：该函数为寄存器操作接口，必须在FPGA人员指导下使用该函数操作地址，对于某些定制功能可通过该函数实现。

44）LVDS_WriteDWord

函数原型：BOOL**LVDS_WriteDWord** (UCHAR ucCard , DWORD dwOffset, DWORD dwData);

说明：该函数提供基本写接口：该函数用于写入指定FPGA寄存器数据。

参数：1）ucCard为板卡索引号。

2）dwOffset为寄存器地址。

3）dwData为写入寄存器的数据。

返回值：TRUE　操作成功

FALSE操作失败

注意：该函数为寄存器操作接口，必须在FPGA人员指导下使用该函数操作地址，对于某些定制功能可通过该函数实现。

5. 操作流程与示例

5.1 发送流程

发送划分为普通发送和 RAM 发送。普通发送即为单次发送，调用一次发送一次。RAM 发送一般为连续发送，可为有限发送，设置完数据后启动发送，FPGA 自动将数据不断发送出去，中间发送过程无需应用软件控制。

5.1.1 普通发送流程

普通发送功能流程如图 4-1 所示，具体流程描述如下：

1)打开板卡：调用 LVDS_Open 函数，该步骤为必须的，只有打开板卡成功后才能继续后续其他操作。

2)进行板卡复位：调用 LVDS_Reset 函数，该步骤为可选，但建议在每次操作前进行该步骤。进行板卡复位后将恢复 FPGA 各寄存器状态到刚上电状态。

3)启动板卡：调用 LVDS_Start 函数，该步骤为必须的，只有启动板卡后才能正常工作。

4)通道参数配置：调用 LVDS_SetCfg 函数，该步骤为可选的，根据需要设置，例如发送波特率、数据位、停止位可在这里进行设置。

5)进行发送帧配置：调用 LVDS_SetFrmCfg 函数，用于配置 FPGA 的帧使能、帧间隔、帧长、帧头和帧尾，该步骤为可选的，需要按照实际情况配置。

6)进行数据发送：通过调用 LVDS_Send 函数实现，该函数为阻塞式，只有等待发送完成或超时失败该函数才会返回。

7)如果继续发送，则转向步骤 6)，如果停止数据发送，则转向步骤 8)。

8)停止并关闭板卡：调用 LVDS_Start(FALSE)函数停止板卡，停止板卡后板卡处于未工作状态，可根据实际需要进行。调用 LVDS_Close 函数释放资源，如果不进行板卡关闭可能会导致资源未释放。

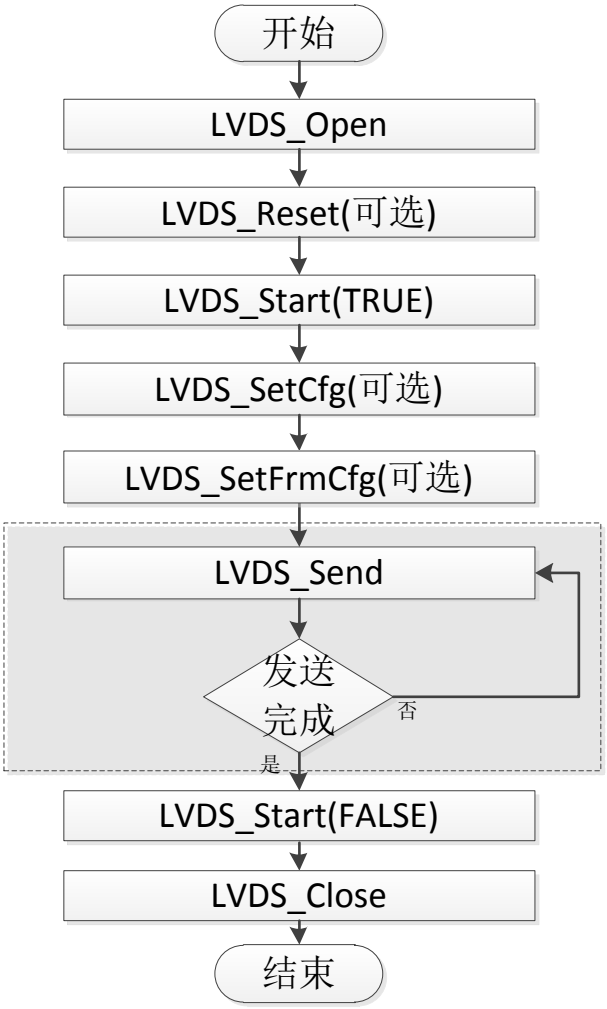


图 4-1 普通发送功能流程图

5.1.2RAM 发送流程

RAM 发送流程配置基本与普通发送流程相同，不同在于设置数据，其流程简单概括为设置帧数据、设置发送帧数、启动发送。

RAM 发送流程如图 4-2 所示，具体流程描述如下：

1)打开板卡：调用 LVDS_Open 函数，该步骤为必须的，只有打开板卡成功后才能继续后续其他操作。

2)进行板卡复位：调用 LVDS_Reset 函数，该步骤为可选，但建议在每次操作前进行该步骤。进行板卡复位后将恢复 FPGA 各寄存器状态到刚上电状态。

3)启动板卡：调用 LVDS_Start 函数，该步骤为必须的，只有启动板卡后才能正常工作。

4)通道参数配置：调用 LVDS_SetCfg 函数，该步骤为可选的，根据需要设置，

例如发送波特率、数据位、停止位可在这里进行设置。

5)进行发送帧配置：调用 LVDS_SetFrmCfg 函数，用于配置 FPGA 的帧使能、帧间隔、帧长、帧头和帧尾，该步骤为可选的，需要按照实际情况配置。

6)进行帧间隔配置：通过调用 LVDS_SetRAMFrmTime 函数实现。

7)进行帧数据配置：通过调用 LVDS_SetRAMFrmData 函数实现，通过帧数据设置同时也设置帧长。

8)重置已发送帧数：通过调用 LVDS_SetRAMFrmData 函数实现，重置当前已发送帧数，便于后续判断发送是否完成（对于有限发送来说）。

9)设置发送帧数：通过调用 LVDS_SetRAMFrmTCnt 函数实现，可设置为无限发送（直到手动停止为止）或有限发送。

10)启动 RAM 发送：通过调用 LVDS_StartRAMSend 函数实现，启动后 FPGA 自动按照设定帧数据和帧间隔发送数据。

11)等待发送完成或停止发：对于有限发送，不断通过调用 LVDS_GetRAMFrmHadTCnt 函数已经发送的帧数，在达到设置发送帧数后转步骤 12）。对于无限发送，直到手动点击停止，转步骤 12）。

12)停止并关闭板卡：调用 LVDS_Start(FALSE)函数停止板卡，停止板卡后板卡处于未工作状态，可根据实际需要进行。调用 LVDS_Close 函数释放资源，如果不进行板卡关闭可能会导致资源未释放。

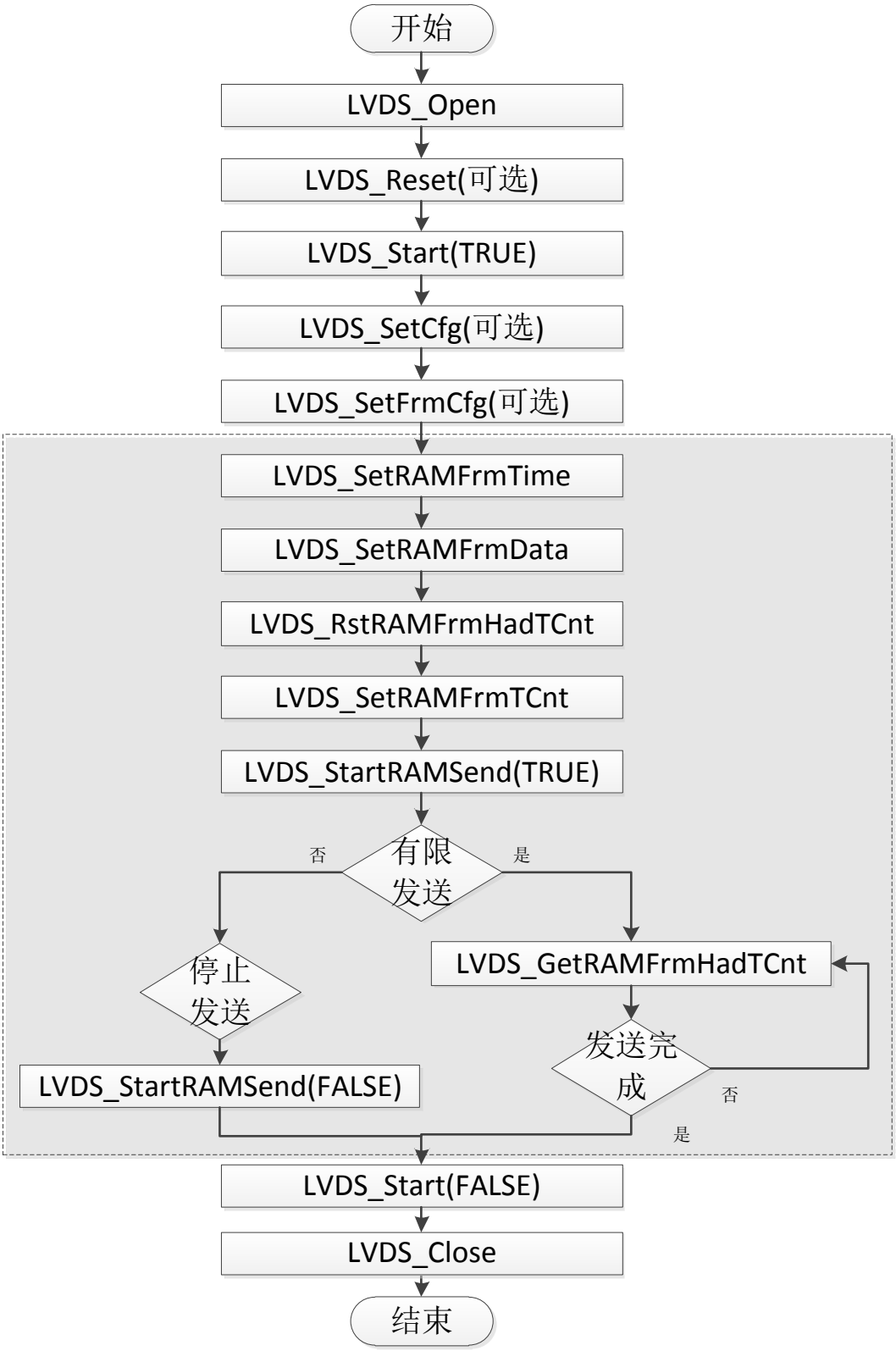


图 4-2RAM 发送功能流程图

5.2 发送示例

数据发送例程如下所示：

```
//以下进行数据发送演示
int _tmain(int argc, _TCHAR* argv[])
{
    BOOL bRet = TRUE;

    if (LVDS_CountCards() <= 0)
    {
        return 0;
    }

    if (TRUE != LVDS_Open(LVDS_CARD_NO))
    {
        return 0;
    }

    bRet &= LVDS_Reset(LVDS_CARD_NO);//板卡复位 可选
    bRet &= LVDS_Start(LVDS_CARD_NO,TRUE);//启动板卡

    /*
    //通道参数配置 根据需要配置
    ST_LVDS_CFG stCfg;
    memset(&stCfg,0,sizeof(stCfg));
    stCfg.dwBaudRate   = 9600;//波特率9600
    stCfg.ucDataBits   = LVDS_DATABITS_8;//8位停止位;
    stCfg.ucParity     = LVDS_PARITY_NO;//无校验
    stCfg.ucParity     = LVDS_STOPBITS_10;//1位停止位
    for (int i = 0; i < LVDS_DATA_CH16; i++)
    {
        bRet &= LVDS_SetCfg(LVDS_CARD_NO,FALSE,LVDS_DATA_CH1+i,&stCfg);//发送通道配置
    }*/

    /*
    //发送帧参数配置 根据需要配置
    ST_LVDS_FRM_CFG stFrmCfg;
    memset(&stFrmCfg,0,sizeof(stFrmCfg));
    stFrmCfg.bEnable    = TRUE;       //帧使能
    stFrmCfg.dwFrmLen   = 0x1000;     //帧长:0x1000字节
    stFrmCfg.dwFrmTime  = 100;        //帧间隔:100个时钟周期
    stFrmCfg.dwFrmHead  = 0xEB90EB90;//帧头:0xEB90EB90
    stFrmCfg.dwFrmEnd   = 0x55AA55AA;//帧尾:0x55AA55AA
    for (int i = 0; i < LVDS_DATA_CH16; i++)
    {
        bRet &= LVDS_SetChFrmCfg(LVDS_CARD_NO,FALSE,LVDS_DATA_CH1+i,&stFrmCfg);//发送通道帧配置
    }*/

    /*********************下面演示普通发送数据(单次发送)*********************/
    BYTE byBuf[8]={0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07};//待发送数据
    DWORD dwSendRet = LVDS_Send(LVDS_CARD_NO,LVDS_DATA_CH1,byBuf,sizeof(byBuf)/sizeof(byBuf[0]),LVDS_WAIT_INFINITE);//超时等待,根据实际需要设置,LVDS_WAIT_INFINITE为一直等待

    /*********************下面演示RAM发送数据(连续发送)*********************/
    if (LVDS_IsSupportRAM(LVDS_CARD_NO))//判断是否支持RAM发送
    {
        BYTE byFrmBuf[10]={0xEB,0x90,0x00,0x01,0x02,0x03,0x04,0x05,0x06,0x07};
        LVDS_SetRAMFrmTime(LVDS_CARD_NO,LVDS_DATA_CH1,100);//设置帧间隔为100us
        LVDS_SetRAMFrmData(LVDS_CARD_NO,LVDS_DATA_CH1,byFrmBuf,sizeof(byFrmBuf)/sizeof(byFrmBuf[0]));//设置帧数据
        LVDS_RstRAMFrmHadTCnt(LVDS_CARD_NO,LVDS_DATA_CH1);//重置已发送帧数 便于统计实际已发送帧数
        LVDS_SetRAMFrmTCnt(LVDS_CARD_NO,LVDS_DATA_CH1,10);//设置发送帧数: 10帧
        LVDS_StartRAMSend(LVDS_CARD_NO,LVDS_DATA_CH1,TRUE);//启动帧发送
        while (LVDS_GetRAMFrmHadTCnt(LVDS_CARD_NO,LVDS_DATA_CH1) < 10)
        {
            //实时查询实际已发送帧数
            Sleep(20);
        }
    }

    //退出程序时 进行板卡停止和板卡关闭操作
    LVDS_Start(LVDS_CARD_NO,FALSE);
    LVDS_Close(LVDS_CARD_NO);
    return 0;
}
```

5.3 接收流程

通道接收流程如图 4-3 所示，具体流程描述如下：

1)打开板卡：调用 LVDS_Open 函数，该步骤为必须的，只有打开板卡成功后才能继续后续其他操作。

2)进行板卡复位：调用 LVDS_Reset 函数，该步骤为可选，但建议在每次操作前进行该步骤。进行板卡复位后将恢复 FPGA 各寄存器状态到刚上电状态。

3)清空 FIFO：调用 LVDS_ClrFIFO 函数，清空接收缓存数据，该步骤为可选的，但建议在进行每次数据接收前进行清空。

4)启动板卡：调用 LVDS_Start 函数，该步骤为必须的，只有启动板卡后才能正常工作。

5)通道参数配置：调用 LVDS_SetCfg 函数，该步骤为可选的，根据需要设置，例如接收波特率、数据位、停止位可在这里进行设置。

6)进行接收帧配置：调用 LVDS_SetFrmCfg 函数，用于配置 FPGA 的帧使能、帧间隔、帧长、帧头和帧尾，该步骤为可选的，需要按照实际情况配置。

7)启动接收:调用 LVDS_QueryStart 函数即可启动数据接收操作。

8)查询接收数据结构个数：调用 LVDS_GetCount 查询接收数据个数，如果该返回值大于 0 则转步骤 9)，否则继续步骤 8)或者退出转步骤 10)。

9)读取数据：通过 LVDS_ReadData 读取接收数据，读取数据结构个数为步骤 8)的返回值，读取处理完毕后转步骤 8)。

10)停止并关闭板卡：调用 LVDS_Start(FALSE)函数停止板卡，停止板卡后板卡处于未工作状态，可根据实际需要进行。调用 LVDS_Close 函数释放资源，如果不进行板卡关闭可能会导致资源未释放。

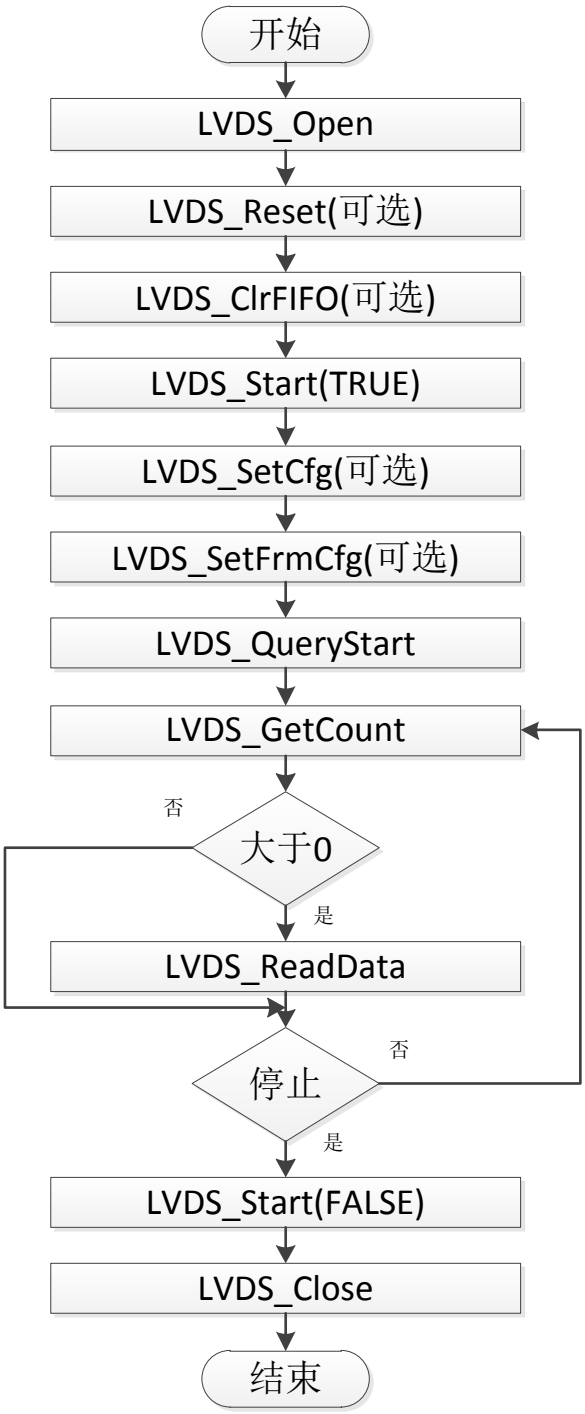


图 4-3 通道接收流程图

5.4 接收示例

数据接收示例如下所示：

```
//以下进行数据接收演示
#define RCV_MAX_ST_CNT  16 //最大接收结构体个数
int _tmain(int argc, _TCHAR* argv[])
{
    BOOL bRet = TRUE;
    ST_LVDS_DATA *pstInfo = NULL;

    if (LVDS_CountCards() <= 0)
    {
        return 0;
    }

    if (TRUE != LVDS_Open(LVDS_CARD_NO))
    {
        return 0;
    }

    pstInfo = (ST_LVDS_DATA*)malloc(sizeof(ST_LVDS_DATA)*RCV_MAX_ST_CNT);
    bRet &= LVDS_Reset(LVDS_CARD_NO);//板卡复位 可选
    bRet &= LVDS_ClrFIFO(LVDS_CARD_NO);//清空缓存
    bRet &= LVDS_Start(LVDS_CARD_NO,TRUE);//启动板卡

    //通道参数配置 根据需要配置
    ST_LVDS_CFG stCfg;
    memset(&stCfg,0,sizeof(stCfg));
    stCfg.dwBaudRate   = 9600;//波特率9600
    stCfg.ucDataBits   = LVDS_DATABITS_8;//8位停止位;
    stCfg.ucParity     = LVDS_PARITY_NO;//无校验
    stCfg.ucParity     = LVDS_STOPBITS_10;//1位停止位
    for (int i = 0; i < LVDS_DATA_CH16; i++)
    {
        bRet &= LVDS_SetCfg(LVDS_CARD_NO,TRUE,LVDS_DATA_CH1+i,&stCfg);//接收通道配置
    }

    //接收帧参数配置 根据需要配置
    ST_LVDS_FRM_CFG stFrmCfg;
    memset(&stFrmCfg,0,sizeof(stFrmCfg));
    stFrmCfg.bEnable    = TRUE;       //帧使能
    stFrmCfg.dwFrmLen   = 0x1000;     //帧长:0x1000字节
    stFrmCfg.dwFrmTime  = 100;        //帧间隔:100个时钟周期
    stFrmCfg.dwFrmHead  = 0xEB90EB90;//帧头:0xEB90EB90
    stFrmCfg.dwFrmEnd   = 0x55AA55AA;//帧尾:0x55AA55AA
    for (int i = 0; i < LVDS_DATA_CH16; i++)
    {
        bRet &= LVDS_SetChFrmCfg(LVDS_CARD_NO,TRUE,LVDS_DATA_CH1+i,&stFrmCfg);//接收通道帧配置
    }

    /*********************下面演示数据接收(演示10次数据接收)*********************/
    DWORD dwRcvCnt = 0;//接收计数
    DWORD dwSTCnt = 0;//查询结构体个数
    LVDS_QueryStart(LVDS_CARD_NO);//启动数据接收
    while (dwRcvCnt < 10)
    {
        dwSTCnt = LVDS_GetCount(LVDS_CARD_NO);//查询缓存中已接收的结构体个数
        if (dwSTCnt > 0)
        {
            dwSTCnt = dwSTCnt > RCV_MAX_ST_CNT?RCV_MAX_ST_CNT:dwSTCnt;//读取个数不能超过RCV_MAX_ST_CNT
            DWORD dwRetCnt = LVDS_ReadData(LVDS_CARD_NO, pstInfo, dwSTCnt);//读取dwSTCnt个结构数据
            for (DWORD i = 0; i < dwRetCnt; i++)
            {
                printf("通道:%d,字节数:%d,数据:",pstInfo[i].emType,pstInfo[i].usLen);
                for (DWORD j = 0; j < pstInfo[i].usLen; j++)
                {
                    printf("0x%02x ",pstInfo[i].ucBuf[j]);
                }
                printf("\r\n");
            }
        }
    }

    //退出程序时 进行板卡停止和板卡关闭操作
    LVDS_Start(LVDS_CARD_NO,FALSE);
    LVDS_Close(LVDS_CARD_NO);
    free(pstInfo);
    return 0;
}
```

6. 应用程序说明

6.1 软件运行

运行 LVDS 板卡测试软件后，出现 6-1 所示“请选择板卡”界面。界面中的板卡数显示当前系统中 LVDS 板卡总数量，选择需要进行测试的板卡号后，点击确定后进入对应板卡的测试界面。如果板卡数显示为 0，请确认板卡已插入插槽，板卡驱动是否正确安装，另外确认是否是管理员用户，如果不是，**右键选择“以管理员身份运行”**LVDS 板卡测试软件，如图 6-2 所示。

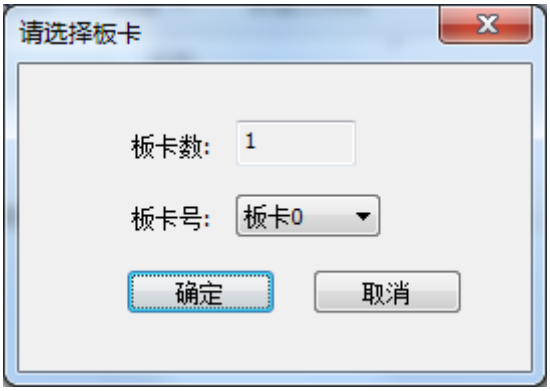


图 6-1 选择板卡

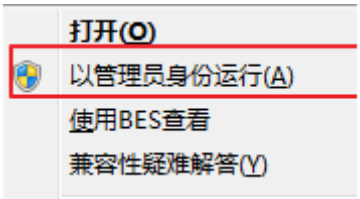


图 6-2 右键选择“以管理员身份运行”

进入后的板卡测试软件界面如图 6-3 所示，其中标题栏包含了当前打开的板卡号，板卡所在总线号、设备号和功能号等信息。如果没有选择板卡或板卡不存在，标题栏会显示打开失败提示信息。

图 6-3LVDS 板卡测试软件界面

软件界面按照功能划分为参数部分、数据接收部分、数据发送部分、日志显示部分、基本读写接口部分、帧计数判断部分、调试信息及设置部分和程序控制部分，划分示意图如图 6-4 所示。

图 6-4 软件界面划分示意图

其中参数部分用于进行通道参数配置，界面如图 6-5 所示，可进行发送通道和接收通道的参数配置和帧配置，参数配置包括波特率、数据位、停止位、校验

位，帧配置包括帧使能、帧长、帧间隔、帧头和帧尾。

图 6-5 通道参数配置界面

下面对其他功能进行详细介绍。

6.2 接收功能

在图 6-3 所示界面点击启动接收按钮，即可启动数据接收，这时如果接收到数据，软件界面将进行接收统计，如图 6-5 所示，其中“已接收”表示自上次重置统计或软件启动后接收的总数据量，以 MB 单位表示。“平均速率”计算方法为当前已接收的总数据量除以启动接收或重置统计到当前的时间，单位为 MB/s。“总字节数”表示自上次重置统计或软件启动接收的总数据量，以字节数表示。“重置统计”可将这些统计信息清零。

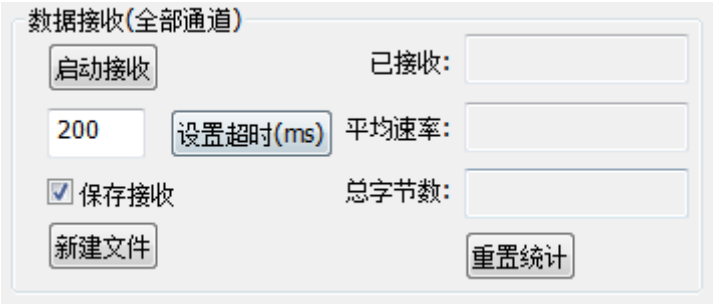


图 5-5 接收到数据后的接收统计

另外，还有一些配置选项可进行选择，如“设置超时”用于设置接收中断超时时间，“保存接收”选项用于设定在接收到数据时是否进行数据保存，“新建文

件”选项用于设置新建数据文件。

对于接收操作，如果接收带有符合特定帧格式，可配合帧计数进行正确性判断。如果想保存接收的数据文件可勾中界面中的“保存接收”选项，存储的数据在软件所在目录下的“数据文件”夹中。

6.3 发送功能

对于发送，板卡测试软件提供了文件发送和帧数据发送功能。

在进行数据发送时，需要先选择使用的发送通道，点击“选择通道…”按钮弹出如图 6-6 所示界面，可选择发送数据的通道。

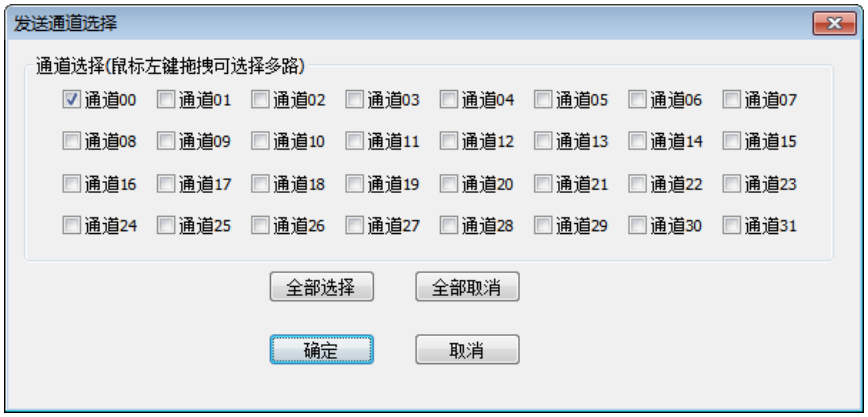


图 6-6 发送通道选择

6.3.1 文件发送

在图 6-3 所示界面点击“选择…”按钮，在弹出的“选择发送文件”对话框中选择待发送文件（支持 bin、dat 和 txt 格式，txt 格式为以空格分隔的 16 进制数据），选择是否循环发送，点击“文件发送”按钮，即可启动发送。

进行数据发送后，软件界面将进行发送统计，如图 6-7 所示，其中“待发送”表示需要发送的总数据量，单位为字节，不限量表示数据连续发送。“已发送”表示自上次重置统计或软件启动后发送的总数据量，以 MB 单位表示。“平均速率”计算方法为当前已发送的总数据量除以启动发送或重置统计到当前的时间，单位为 MB/s。“总字节数”表示自上次重置统计或软件启动发送的总数据量，以字节数表示。“重置统计”可将这些统计信息清零。

图 6-7 发送数据统计信息

“循环发送”设置项表示在文件发送到文件尾后，是否选择从文件头继续发送。如果不勾中，则发送完当前文件大小的数据量后自动停止，如果勾中，则一直连续发送，直到手动点击停止发送为止。

如果想查看实际发送的文件内容，可在界面中勾中“保存发送”选项，保存的数据文件在软件所在目录下的“数据文件”夹中。

6.3.2 帧发送

板卡测试软件还提供了按照设定帧格式生成固定数进行发送的功能，在图 6-3 所示界面的固定数发送界面部分，选择相应的发送模式，点击“帧发送”按钮即可启动帧数据发送，启动发送后在发送统计有相应的数据更新。

帧发送提供几种发送模式，包括连续、定量、定帧。连续模式是指按照设定帧格式持续进行数据发送，直到手动点击停止发送为止。定量模式是指按照设定字节数进行数据发送，数据内容使用帧数据进行填充，例如帧长为 4194304，设置定量字节数为 4194320，则发送数据为第一帧的整帧和第二帧的前 16 个字节。定帧模式是指按照设定帧数进行数据发送，发送的数据量取决于设定的帧长，例如帧长为 4194304，设置帧个数为 1000，则发送的数据量为 4194304000 字节。

帧格式可进行设置，在界面中点击“帧格式...”按钮弹出图 6-8 所示帧格式设置对话框，可修改帧头、帧长、帧计数起始、帧计数结束和帧尾，各参数值可进行修改，但占用字节数和顺序已经固定不可修改，例如帧头默认为 0x1ACFFC1D，占用 4 字节，可修改为 0xEB90EB90，内容可变，但占用 4 字节数不能修改。

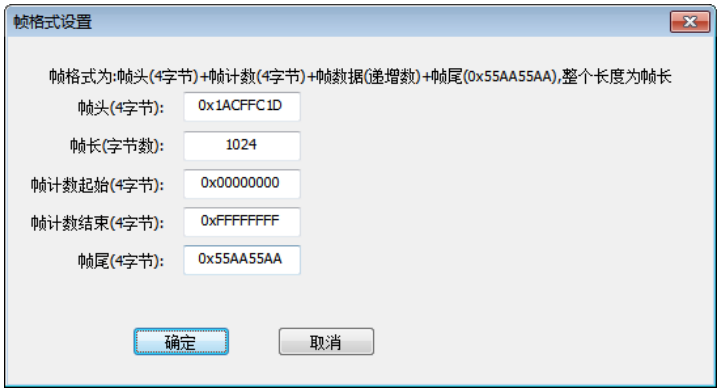


图 6-8 帧格式设置对话框

如果想查看实际发送的帧数据，可在界面中勾中“保存发送”选项，保存的数据文件在软件所在目录下的“数据文件”夹中。

6.4 日志显示

日志显示用于显示所有操作信息和提示信息，同时保存日志文件，便于后续查看，界面的日志显示可通过“日志显示”选项进行开关。

6.5 基本读写接口

基本读写接口部分提供了基本读写接口与软件的配置功能。

读取版本按钮可以查看本板卡的 FPGA 版本信息。

写入和读取按钮，可以对指定地址读取数据或写入数据，主要用于调试测试时的验证，也可用于后续扩展。地址和数据均为 16 进制。

6.6 调试接口

调试接口主要用于问题排查和信息查看，是软件的辅助功能，界面如图 6-9 所示。

图 6-9 调试接口界面

板卡启动和板卡停止按钮用于手动控制板卡启动状态，在软件启动后会自动进行板卡启动，但有时为了进行测试可手动进行板卡启动和板卡停止。

板卡复位用于进行板卡复位，恢复 FPGA 到刚上电默认状态。

清空缓存用于清空接收缓存，包括 FPGA 的接收 FIFO 和驱动的接收循环缓存。

配置序列用于按照 config.xml 的配置指令自动进行寄存器的读写，配置序列格式如图 6-10 所示，其中 type 为 r 表示读，为 w 表示写，adr 表示 16 进制寄存器地址，data 为 16 进制数据（对于 type 为 r 忽略 data）。例如图中所示序列含义为，数据 0x01 写入地址 0x04，读取地址 0x08 数据（忽略数据内容），数据 0x02 写入地址 0x0C，数据 0x03 写入地址 0x10。

```
<ORDER    name="配置序列"    type="w" adr="0x04" data="0x01"/>
<ORDER    name="配置序列"    type="r" adr="0x08" data=""/>
<ORDER    name="配置序列"    type="w" adr="0x0C" data="0x02"/>
<ORDER    name="配置序列"    type="w" adr="0x10" data="0x03"/>
```

图 6-10 配置序列内容

校验版本用于验证当前所用的动态库是否与当前安装的板卡驱动以及板卡的 FPGA 程序匹配正确。

查询发送量用于查询指定通道 FPGA 统计的实际发送字节数，可通过重置发送量进行重置。

各调试寄存器状态显示了调试寄存器的值，包括启动状态、程序版本、设备 ID、编译时间、中断使能和时钟频率，通过点击调试信息按钮进行查询。

6.7 帧计数判断

勾中使能帧计数判断，可对接收的数据进行帧计数判断，在发生帧计数跳变时，日志信息会有提示信息如图 6-11 所示。

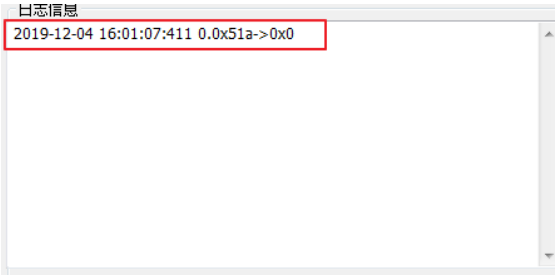


图 6-11 帧计数跳变提示信息

使用该功能的前提条件是接收的数据带固定帧头（4 字节）的，并且帧长固定，帧计数（4 字节）紧跟帧头后面，即帧格式如表 6-1 所示。判断帧计数连续以该次帧计数比上次帧计数大 1 计算的。

表 6-1 帧格式

帧头(4 字节)	帧长(4 字节)	帧数据(定长)

帧计数判断的帧头、帧长可进行修改，在帧头编辑框和帧长编辑框输入实际的帧头和帧长，点击设置，即可修改，之后帧计数判断按照修改后的帧信息进行判断。

6.8 程序控制

程序控制部分用于进行一些软件控制功能。

清空日志用于清空界面的日志信息。

打开目录用于打开数据文件所在位置。

退出程序用于关闭软件。

显示日志用于设定是否显示日志，勾中为显示，不勾中为不显示。

计算速率用于设定是否进行发送速率和接收速率的计算，勾中为计算，不勾中为不计算。

6.9 软件版本

查看测试软件编译时间和版本。点击图 6-12 所示界面的标题栏的板卡图标

可以查看当前应用程序的编译时间，该软件的编译信息如图 6-13 所示。

图 6-12 点击标题栏板卡图标

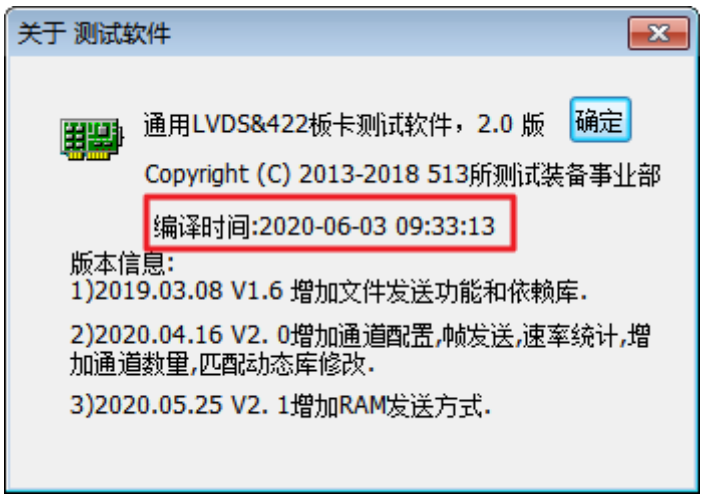


图 6-13 应用软件编译时间

常见问题

1.板卡驱动安装不成功？

1）硬件板卡是否存在。

2）板卡类型是否匹配（VendorID 和 DeviceID 是否与 Inf 文件中一致）。

3）如果为 64 位系统，请确认系统是否处于测试模式。

4）重启下系统试下。

2.打开测试软件识别不到板卡？

1）请确认在设备管理器中能否查看到该板卡。

2）驱动是否安装成功，没有显示黄色叹号。

3）确认是否以管理员身份运行的测试软件。

3.各功能不好用或版本校验失败？

1）请确认 FPGA 是否按照约定协议进行实现，具体查看通用 LVDS 协议